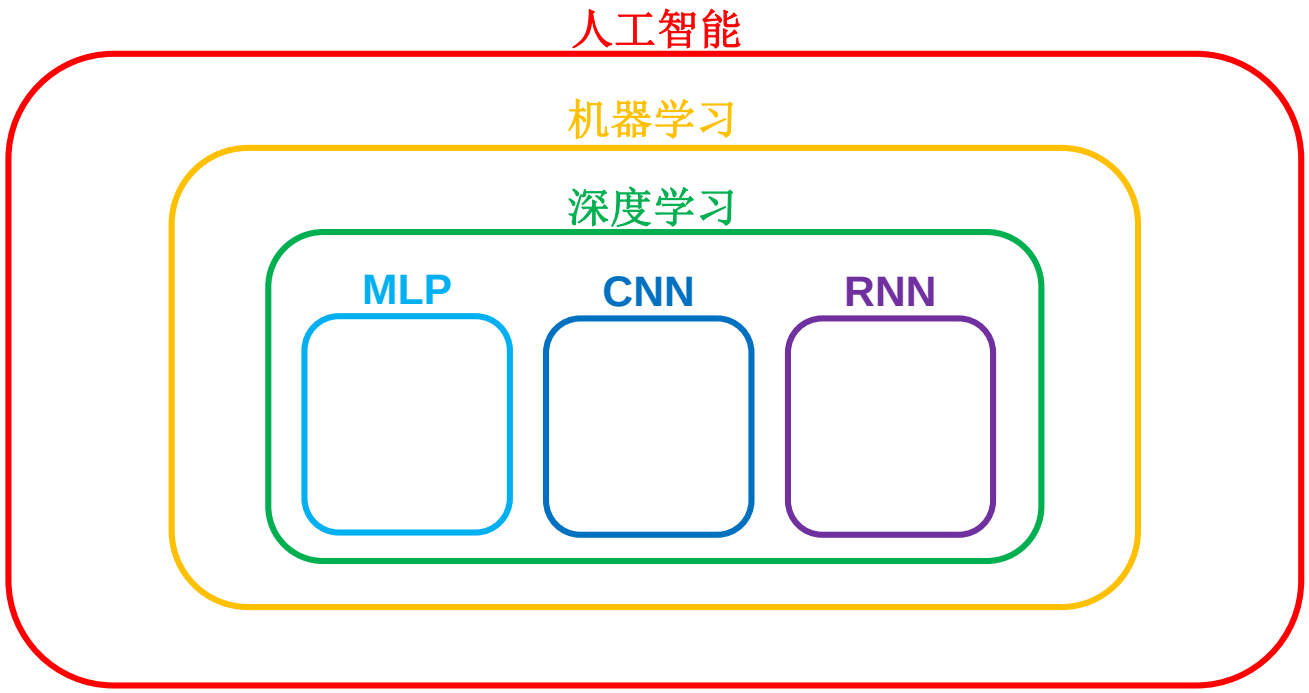
深度学习

什么是深度学习？

【Wikipedia的定义】 **Deep learning** is part of a broader family of machine learning methods based on **artificial neural networks**.



14.1 深度学习发展简史
14.2 多层感知机
14.3 损失函数
14.4 优化算法
14.5 案例：手写数字识别
14.6 深度学习技巧
14.7 卷积神经网络
14.8 案例：图像识别

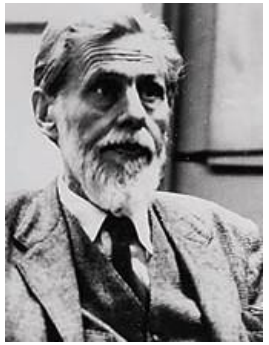
深度学习发展简史

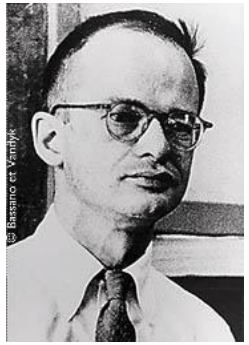
第一阶段

- 1943年，心理学家McCulloch和数学家Pitts提出第一个神经元模型，即M-P模型，并从原理上证明了人工神经网络能够计算任何算术和逻辑函数

A Logical Calculus of Ideas Immanent in Nervous Activity

神经活动中思想内在性的逻辑演算(神经网络的开山之作)



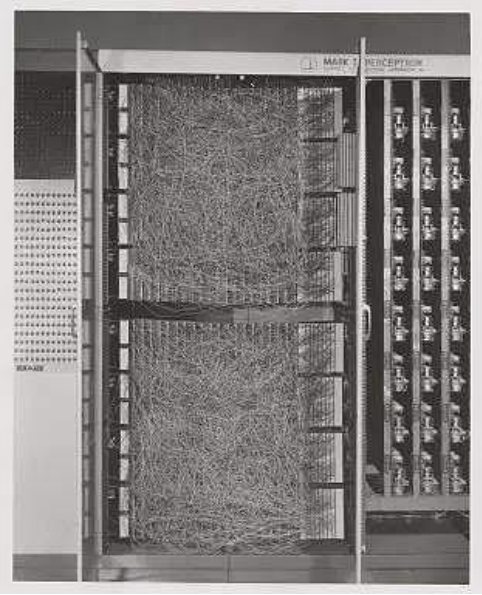


神经科学 + 数理逻辑 = 神经网络

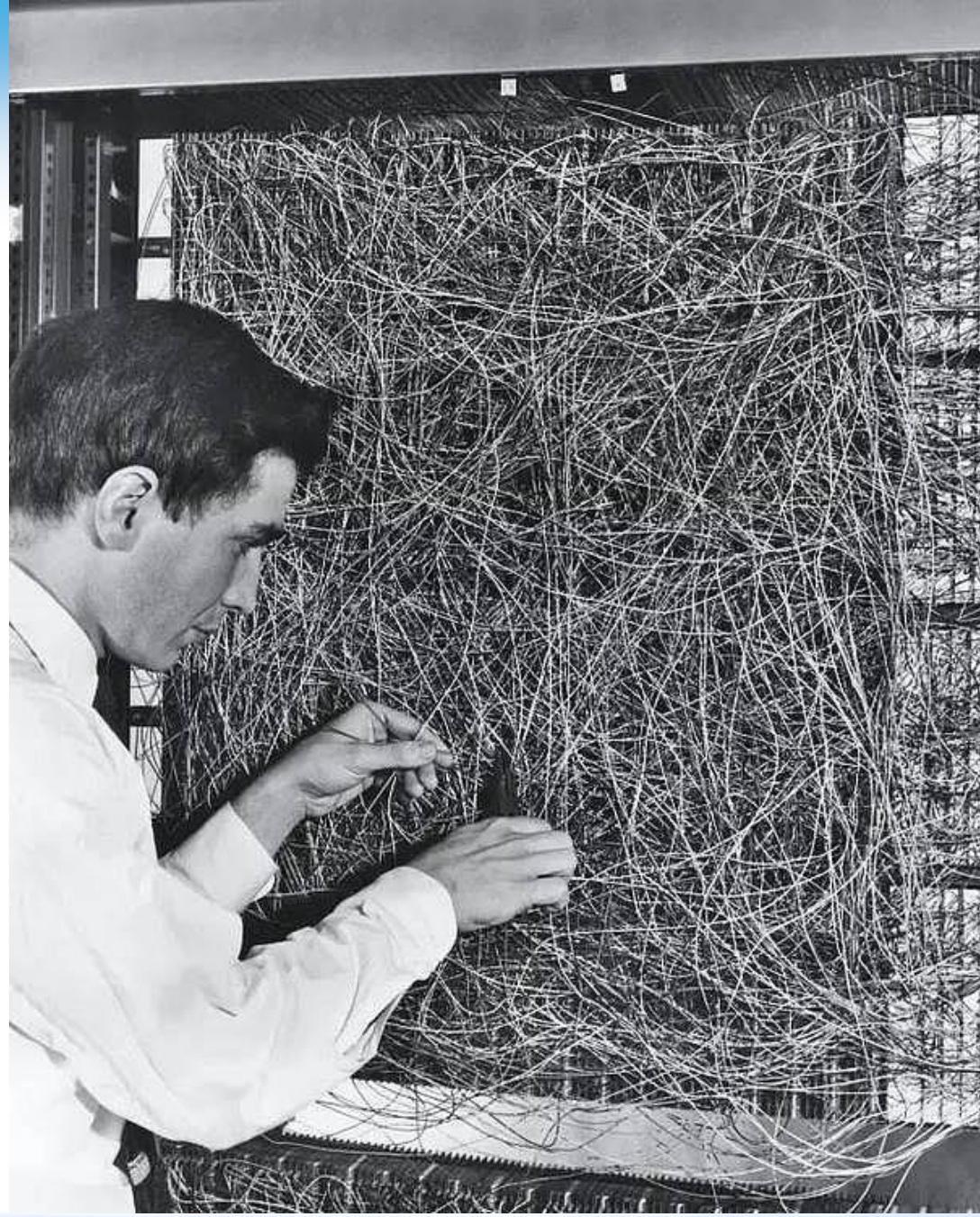
- 1949年，Hebb 发表《The Organization of Behavior》一书，提出生物神经元学习的机理，即Hebb学习规则

  人脑神经细胞的突触上的强度是可以变化的

- 1958年，Rosenblatt 提出感知机网络（Perceptron）模型和其学习规则



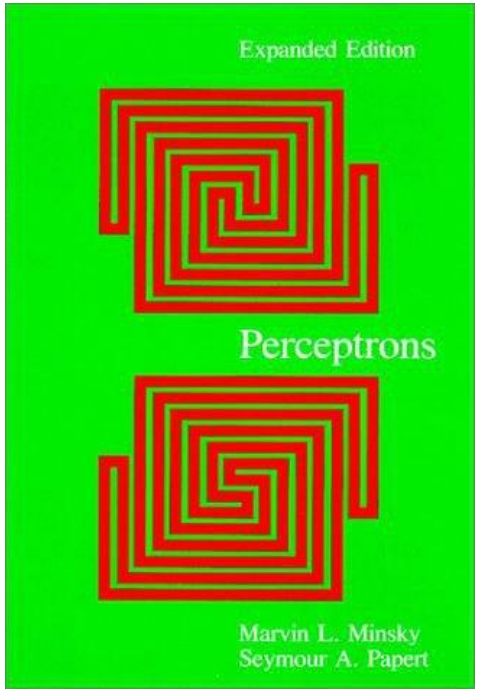
在美国纽约市水牛城的康奈尔航空实验室内，一名工作人员在调试罗森布拉特发明的人工神经网络系统。（摄于1960年）



- 1969年，Minsky(1969年获图灵奖)和Papert 发表《Perceptrons》一书，指出单层神经网络不能解决非线性问题，多层网络的训练算法尚无希望．这个论断导致神经网络进入低谷

长达近二十年的低潮

史称“人工智能冬天（AI Winter）”

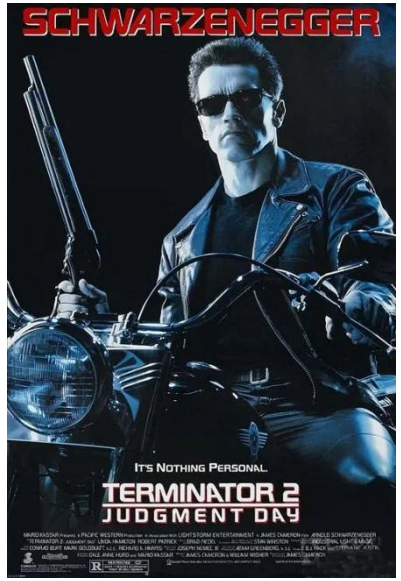


第二阶段

- 1982年，物理学家Hopfield提出了一种具有联想记忆、优化计算能力的递归网络模型，即Hopfield 网络
- 1986年，Rumelhart 等编辑的著作《Parallel Distributed Processing : Explorations in the Microstructures of Cognition》报告了反向传播算法

Rumelhart D E, Hinton G E, Williams R J. Learning representations by back-propagating errors[J]. nature, 1986, 323(6088): 533.

- 1987年，IEEE 在美国加州圣地亚哥召开第一届神经网络国际会议（ICNN）
- 20世纪90年代初，伴随统计学习理论和SVM的兴起，神经网络由于理论不够清楚，试错性强，难以训练，再次进入低谷



在1991年《终结者2》电影中，施瓦辛格扮演的“终结者”机器人也说：My CPU is a neural-net processor...a learning computer.

第三阶段

- 2006年，Hinton提出了深度信念网络(DBN)，通过“预训练+微调”使得深度模型的最优化变得相对容易

“... provided that computers were fast enough, data sets were big enough, and the initial weights were close enough to a good solution. All three conditions are now satisfied.”

materials are identical for all configurations. The blue bars in Fig. 1 summarize the measured SHG signals. For excitation of the *LC* resonance in Fig. 1A (horizontal incident polarization), we find an SHG signal that is 500 times above the noise level. As expected for SHG, this signal closely scales with the square of the incident power (Fig. 2A). The polarization of the SHG emission is nearly vertical (Fig. 2B). The small angle with respect to the vertical is due to deviations from perfect mirror symmetry of the SRRs (see electron micrographs in Fig. 1). Small detuning of the *LC* resonance toward smaller wavelength (i.e., to 1.3-μm wavelength) reduces the SHG signal strength from 100% to 20%. For excitation of the Mie resonance with vertical incident polarization in Fig. 1D, we find a small signal just above the noise level. For excitation of the Mie resonance with horizontal incident polarization in Fig. 1C, a small but significant SHG emission is found, which is again po-

Reducing the Dimensionality of Data with Neural Networks

G. E. Hinton* and R. R. Salakhutdinov

High-dimensional data can be converted to low-dimensional codes by training a multilayer neural network with a small central layer to reconstruct high-dimensional input vectors. Gradient descent can be used for fine-tuning the weights in such "autoencoder" networks, but this works well only if the initial weights are close to a good solution. We describe an effective way of initializing the weights that allows deep autoencoder networks to learn low-dimensional codes that work much better than principal components analysis as a tool to reduce the dimensionality of data.

Dimensionality reduction facilitates the classification, visualization, communication, and storage of high-dimensional data. A simple and widely used method is principal components analysis (PCA), which finds the directions of greatest variance in the data set and represents each data point by its coordinates along each of these directions. We describe a nonlinear generalization of PCA that uses an adaptive, multilayer "encoder" network

Department of Computer Science, University of Toronto, 6 King's College Road, Toronto, Ontario M5S 3G4, Canada.

*To whom correspondence should be addressed; E-mail: hinton@cs.toronto.edu

gradients are easily obtained by using the chain rule to backpropagate error derivatives first through the decoder network and then through the encoder network (*1*). The whole system is

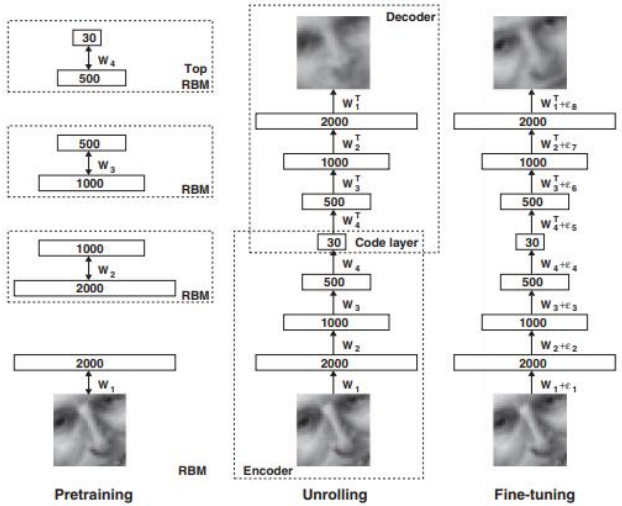


Fig. 1. Pretraining consists of learning a stack of restricted Boltzmann machines (RBMs), each having only one layer of feature detectors. The learned feature activations of one RBM are used as the "data" for training the next RBM in the stack. After the pretraining, the RBMs are "unrolled" to create a deep autoencoder, which is then fine-tuned using backpropagation of error derivatives.

hidden layers (*2–4*). With large initial weights, autoencoders typically find poor local minima; with small initial weights, the gradients in the early layers are tiny, making it infeasible to train autoencoders with many hidden layers. If the initial weights are close to a good solution, gradient descent works well, but finding such initial weights requires a very different type of algorithm that learns one layer of features at a time. We introduce this "pretraining" procedure for binary data, generalize it to real-valued data, and show that it works well for a variety of data sets.

An ensemble of binary vectors (e.g., images) can be modeled using a two-layer network called a "restricted Boltzmann machine" (RBM) (*5*, *6*) in which stochastic, binary pixels are connected to stochastic, binary feature detectors using symmetrically weighted connections. The pixels correspond to "visible" units of the RBM because their states are observed; the feature detectors correspond to "hidden" units. A joint configuration ($\mathbf{v}$, $\mathbf{h}$) of the visible and hidden units has an energy (*7*) given by

$$E(\mathbf{v},\mathbf{h}) = -\sum_{i\in\text{pixels}} b_i v_i - \sum_{j\in\text{features}} b_j h_j - \sum_{i,j} v_i h_j w_{ij} \quad (1)$$

where v_i and h_j are the binary states of pixel i and feature j, b_i and b_j are their biases, and w_{ij} is the weight between them. The network assigns a probability to every possible image via this energy function, as explained in (*8*). The probability of a training image can be raised by

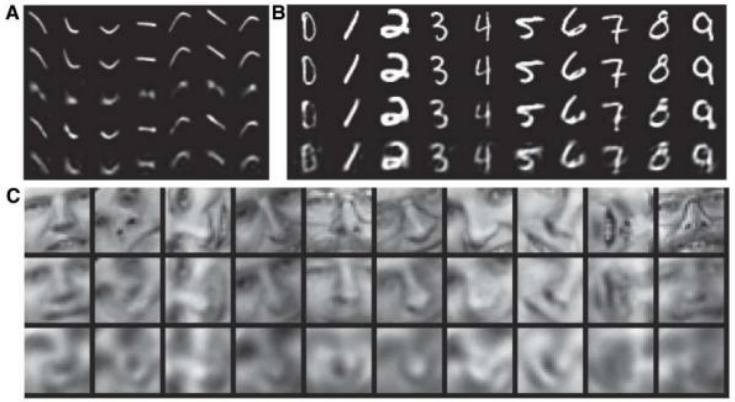
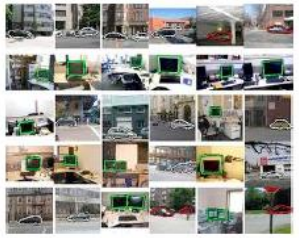


Fig. 2. (**A**) Top to bottom: Random samples of curves from the test data set; reconstructions produced by the six-dimensional deep autoencoder; reconstructions by "logistic PCA" (*8*) using six components; reconstructions by logistic PCA and standard PCA using 18 components. The average squared error per image for the last four rows is 1.44, 7.64, 2.45, 5.90. (**B**) Top to bottom: A random test image from each class; reconstructions by the 30-dimensional autoencoder; reconstructions by 30-dimensional logistic PCA and standard PCA. The average squared errors for the last three rows are 3.00, 8.01, and 13.87. (**C**) Top to bottom: Random samples from the test data set; reconstructions by the 30-dimensional autoencoder; reconstructions by 30-dimensional PCA. The average squared errors are 126 and 135.

- 2012年，Hinton 组参加ImageNet 竞赛，使用 CNN 模型以超过第二名10个百分点的成绩夺得当年竞赛的冠军

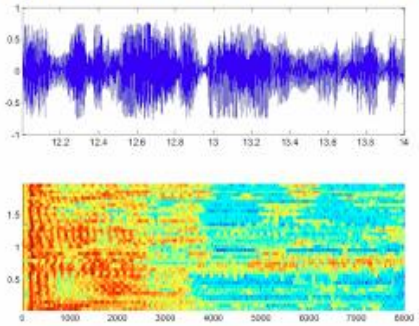
- 伴随云计算、大数据时代的到来，计算能力的大幅提升，使得深度学习模型在计算机视觉、自然语言处理、语音识别等众多领域都取得了较大的成功

Images & Video



Text & Language

Speech & Audio



深度学习发展简史（一）

1943年，Warren McCullock和Walter Pitts提出第一个**脑神经元**的抽象模型

1958年，Frank Rossenblatt基于MP神经元模型提出第一个**感知器**学习算法

1969年，Marvin Minsky和Seymour Papert出版了《Perceptron》一书，分析了感知器模型的局限性

1986年，D.E. Rumerlhart，G.E. Hinton和R.J. Williams发表论文，阐释并推广了**反向传播算法**（Back Propagation, BP）

深度学习发展简史（二）

2006年，Hinton等人提出一种可以实现快速学习的**深度**信念网络（DBN）

2009年，**GPU**被用于训练深度神经网络

2012年，AlexNet在ImageNet**图像识别**竞赛中达到83.6%的Top5精度

2015年，微软的ResNet以96.43%的Top5精度，超过人类水平（94.9%）

2016年，AlphaGo击败**围棋**世界冠军Lee Sedol

2017年，AlphaGo Zero以100:0击败AlphaGo

2017年，IBM将**语音识别**错误率降到5.5%（人类5.1%）

2022年，OpenAI发布ChatGPT

2019年3月27日，ACM宣布：

Geoffrey Hinton, Yann LeCun, Yoshua Bengio

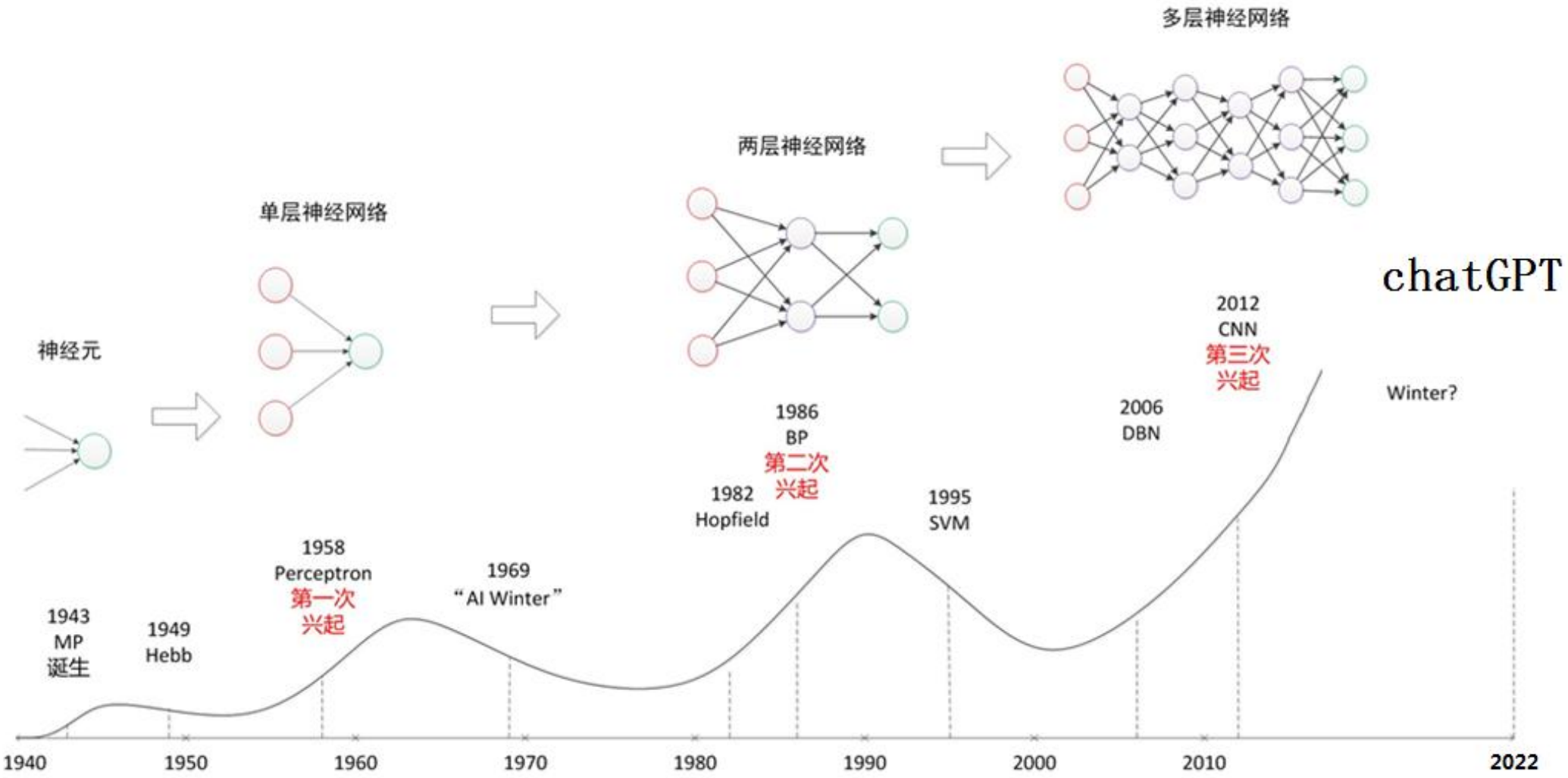
因对深度学习的卓越贡献获得图灵奖

2024年诺贝尔物理学奖

人工神经网络
实现机器学习

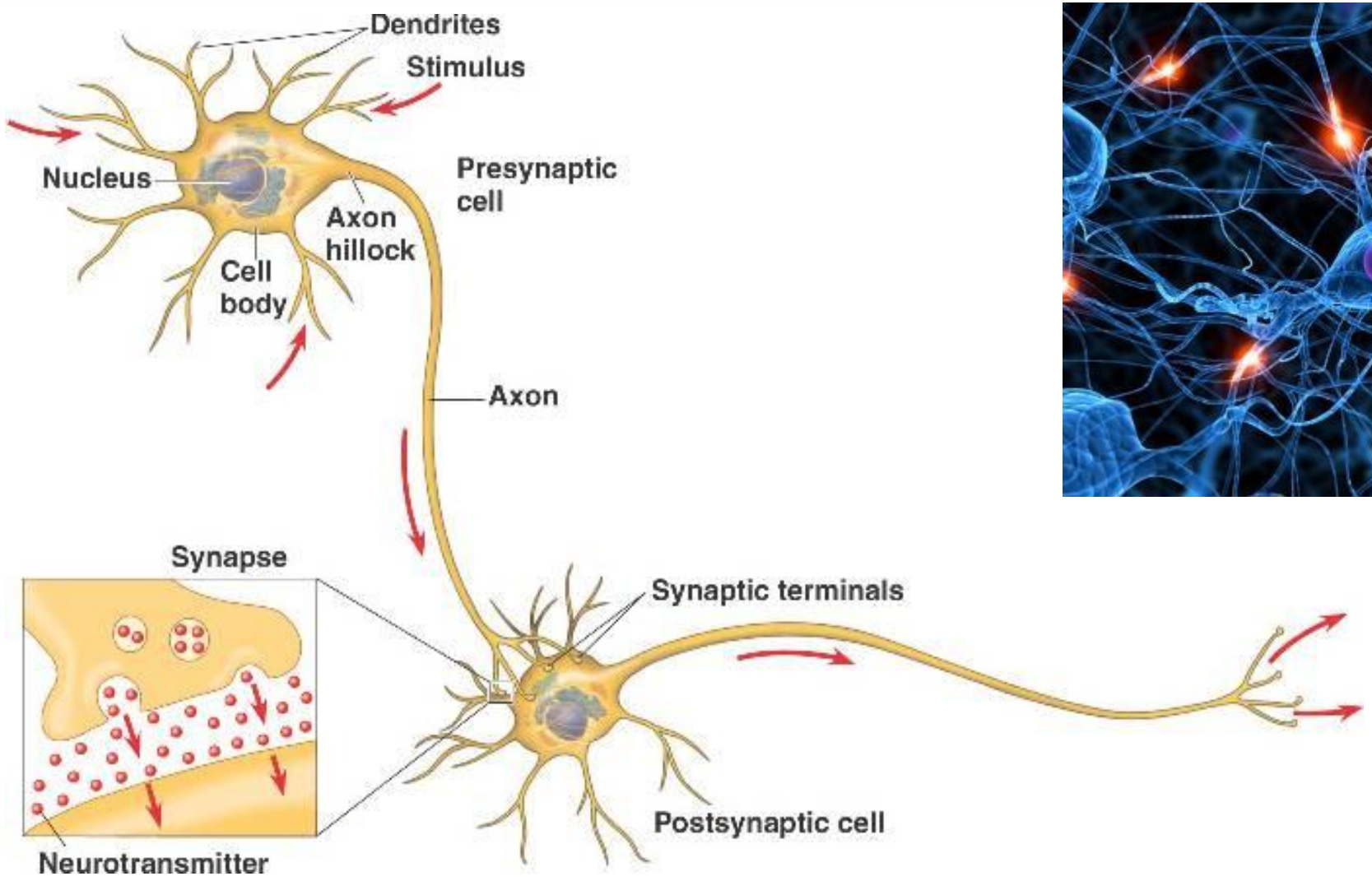
2024年诺贝尔化学奖

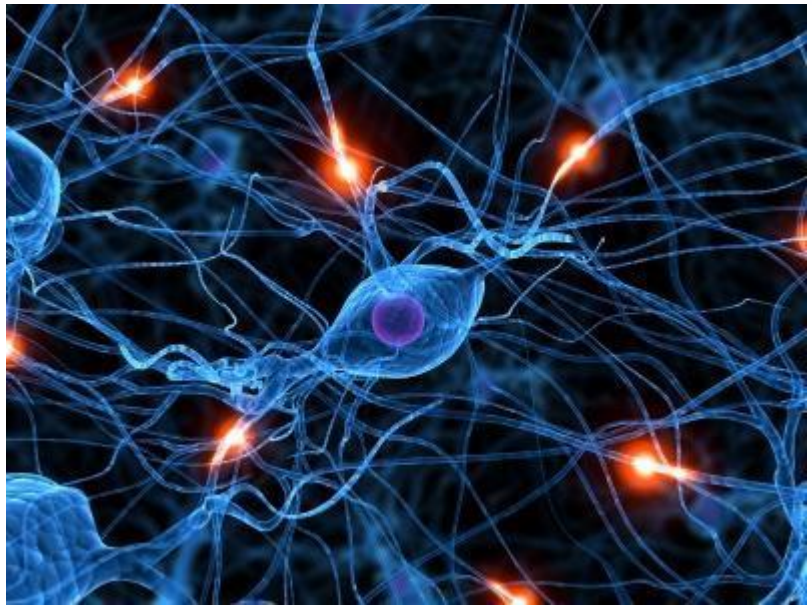
AI推动蛋白质结构预测的革命


多层神经网络
两层神经网络
单层神经网络
神经元
chatGPT
2012
CNN
第三次
兴起
Winter?
2006
DBN
1986
BP
第二次
兴起
1982
Hopfield
1995
SVM
1958
Perceptron
第一次
兴起
1969
"AI Winter"
1943
MP
诞生
1949
Hebb
1940
1950
1960
1970
1980
1990
2000
2010
2022

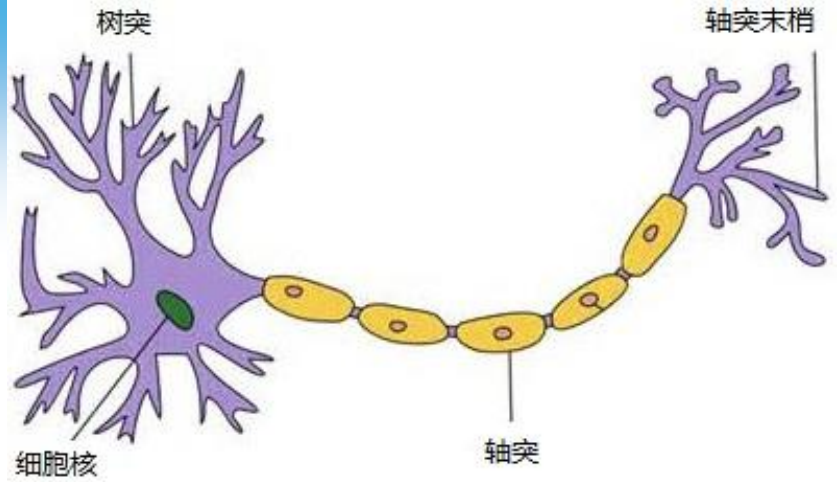
14.1 深度学习发展简史
14.2 多层感知机
14.3 损失函数
14.4 优化算法
14.5 案例：手写数字识别
14.6 深度学习技巧
14.7 卷积神经网络
14.8 案例：图像识别

人脑神经元


Dendrites
Stimulus
Nucleus
Presynaptic cell
Axon hillock
Cell body
Axon
Synapse
Synaptic terminals
Postsynaptic cell
Neurotransmitter



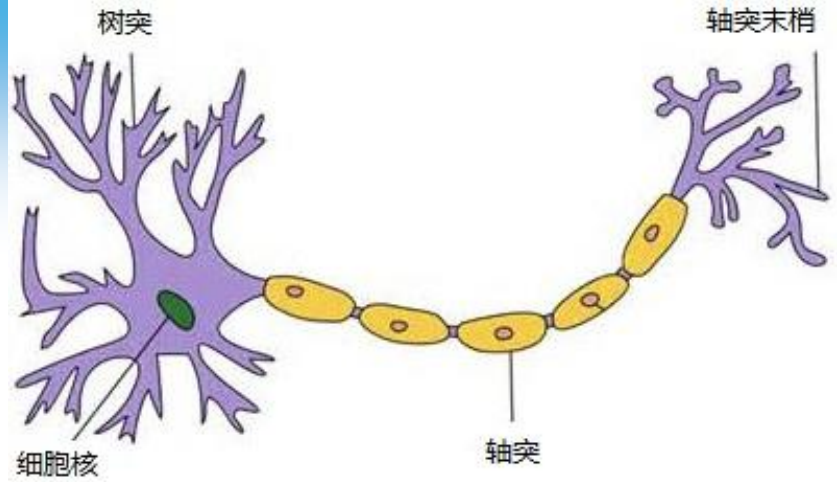
神经元模型

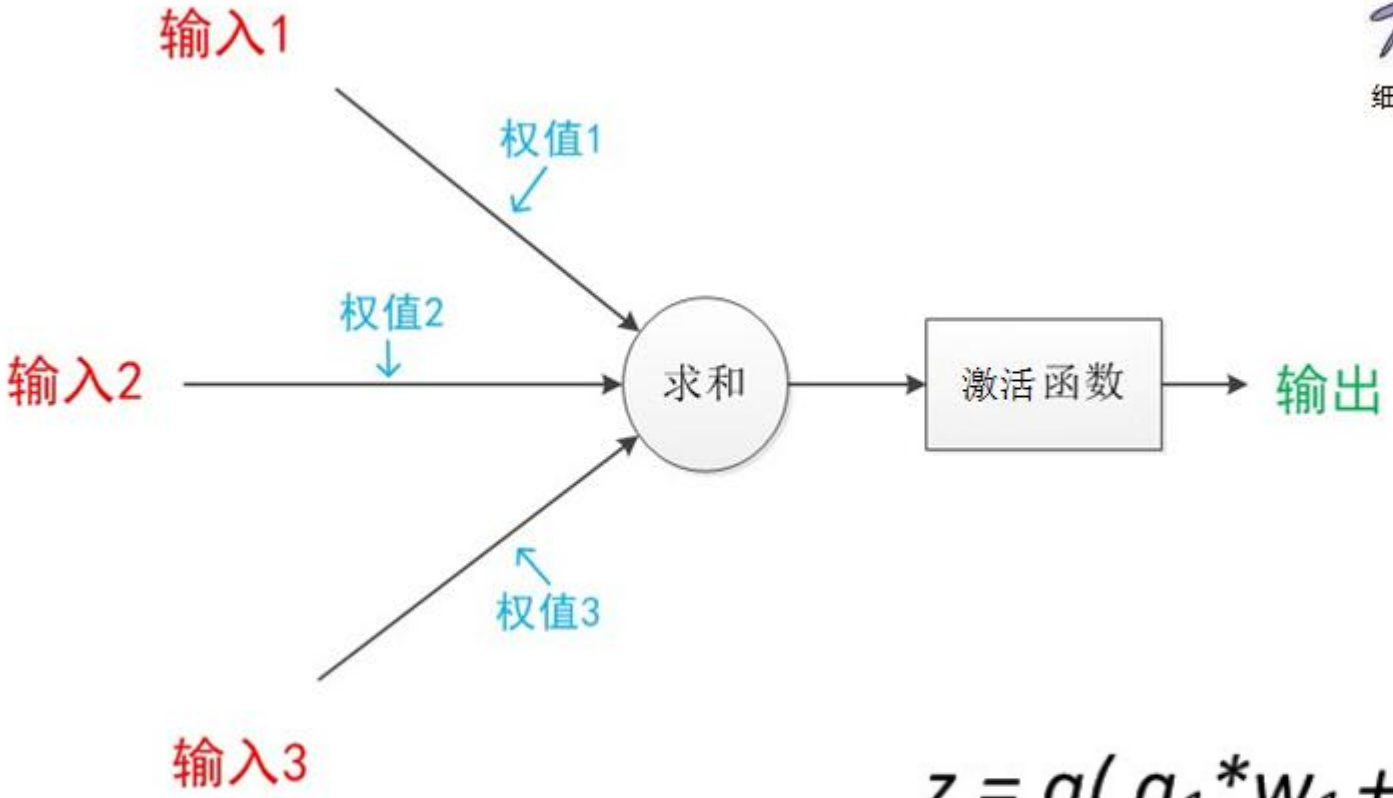


“神经网络是由具有适应性的简单单元组成的广泛并行互联的网络，它的组织能够模拟生物神经系统对真实世界物体所作出的反应”

[Kohonen, 1988, Neural Networks 创刊号]

- 神经元模型即上述定义中的“简单单元”，是神经网络的基本成分
- 生物神经网络：每个神经元与其他神经元相连，当它“兴奋”时，就会向相连的神经元发送化学物质，从而改变这些神经元内的电位；如果某神经元的电位超过一个“阈值”，那么它就会被激活，即“兴奋”起来，向其它神经元发送化学物质

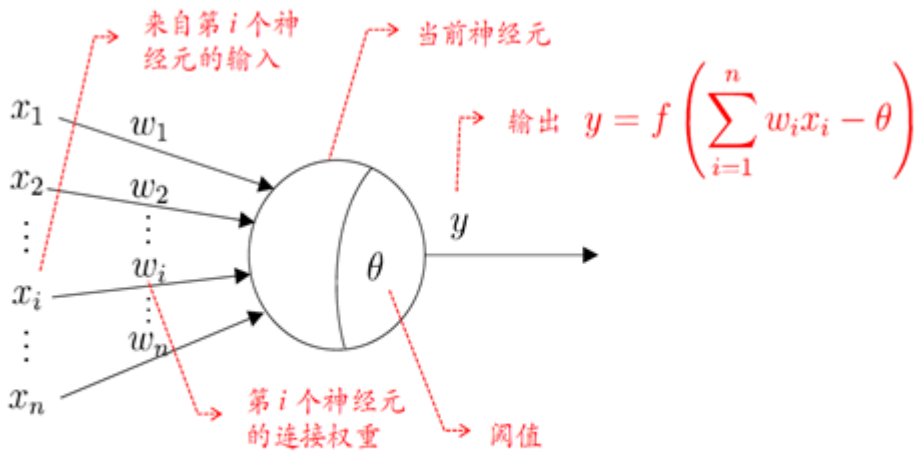




$$z = g(a_1 * w_1 + a_2 * w_2 + a_3 * w_3)$$

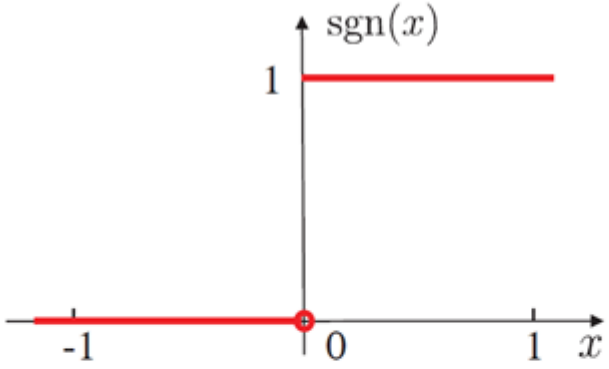
M-P 神经元模型 [McCulloch and Pitts, 1943]

- 输入：来自其他n个神经元传递过来的输入信号
- 处理：输入信号通过带权重（**预设**）的连接进行传递，神经元接收到总输入值将与神经元的阈值（**预设**）进行比较
- 输出：通过激活函数的处理以得到输出



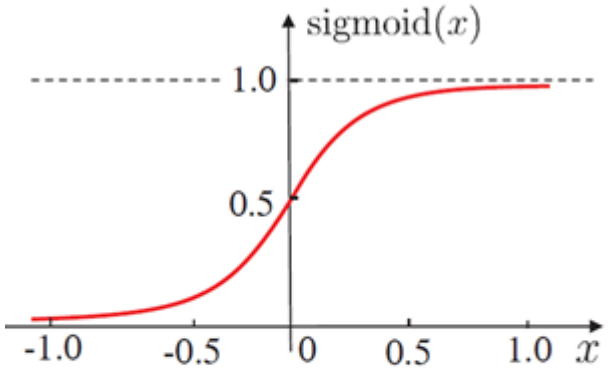
M-P 神经元模型

激活函数



$$\mathrm{sgn}(x) = \begin{cases} 1, & \text{if } x \geq 0; \\ 0, & \text{if } x < 0. \end{cases}$$

(a) 阶跃函数



$$\mathrm{sigmoid}(x) = \frac{1}{1+e^{-x}}$$

(b) Sigmoid 函数

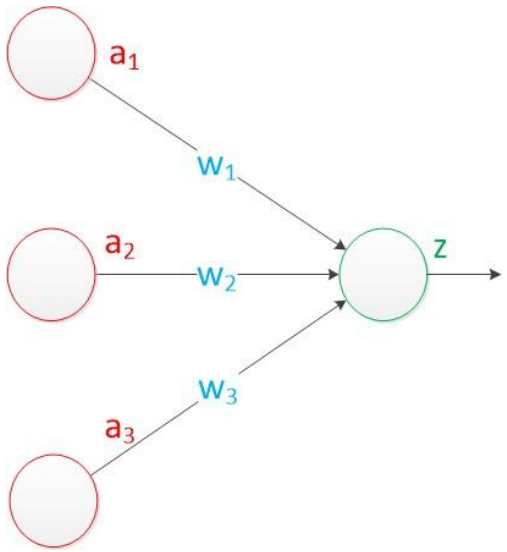
典型的神经元激活函数

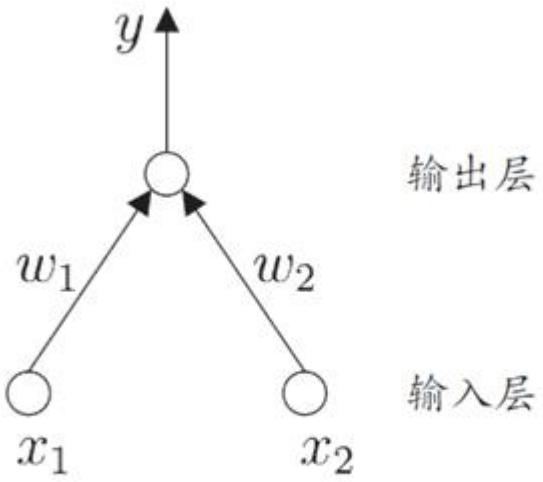
- 理想激活函数是阶跃函数，0表示抑制神经元而1表示激活神经元
- 阶跃函数具有不连续、不光滑等不好的性质

常用的是 Sigmoid 函数

感知机模型

- 感知机由两层神经元组成，输入层接受外界输入信号传递给输出层，输出层是M-P神经元（阈值逻辑单元）





两个输入神经元的感知机网络结构示意图

- 给定训练数据集，权重 $w_i(i=1,2,\cdots,n)$ 与阈值 θ 可以通过学习得到
- 感知机学习规则

对训练样例 $(\boldsymbol{x},y)$ ，若当前感知机的输出为 $\hat{y}$ ，则感知机**权重调整**规则为：

$$w_i \leftarrow w_i + \triangle w_i$$
$$\triangle w_i = \eta(y-\hat{y})x_i$$

其中 $\eta \in (0,1)$ 称为学习率。

若感知机对训练样例 $(\boldsymbol{x},y)$ 预测正确，则感知机不发生变化；否则根据**错误程度**进行权重的调整.

若两类模式线性可分，则感知机的学习过程一定会收敛；否则感知机的学习过程将会发生震荡

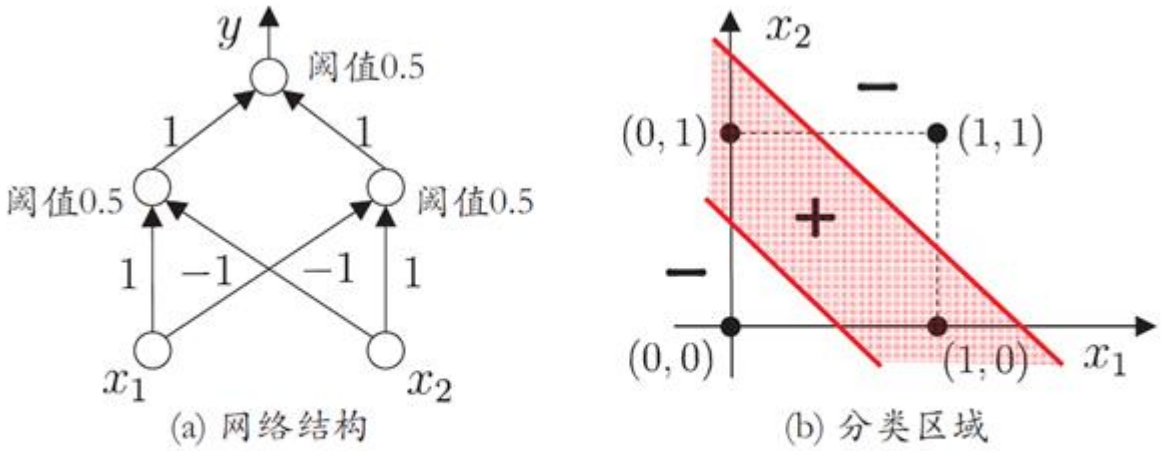
[Minsky and Papert, 1969]

单层感知机的学习能力非常有限，只能解决线性可分问题

- 事实上，与、或、非问题是线性可分的，因此感知机学习过程能够求得适当的权值向量．而异或问题不是线性可分的，感知机学习不能求得合适解
- 对于非线性可分问题，如何求解？

多层感知机

解决异或问题的两层感知机



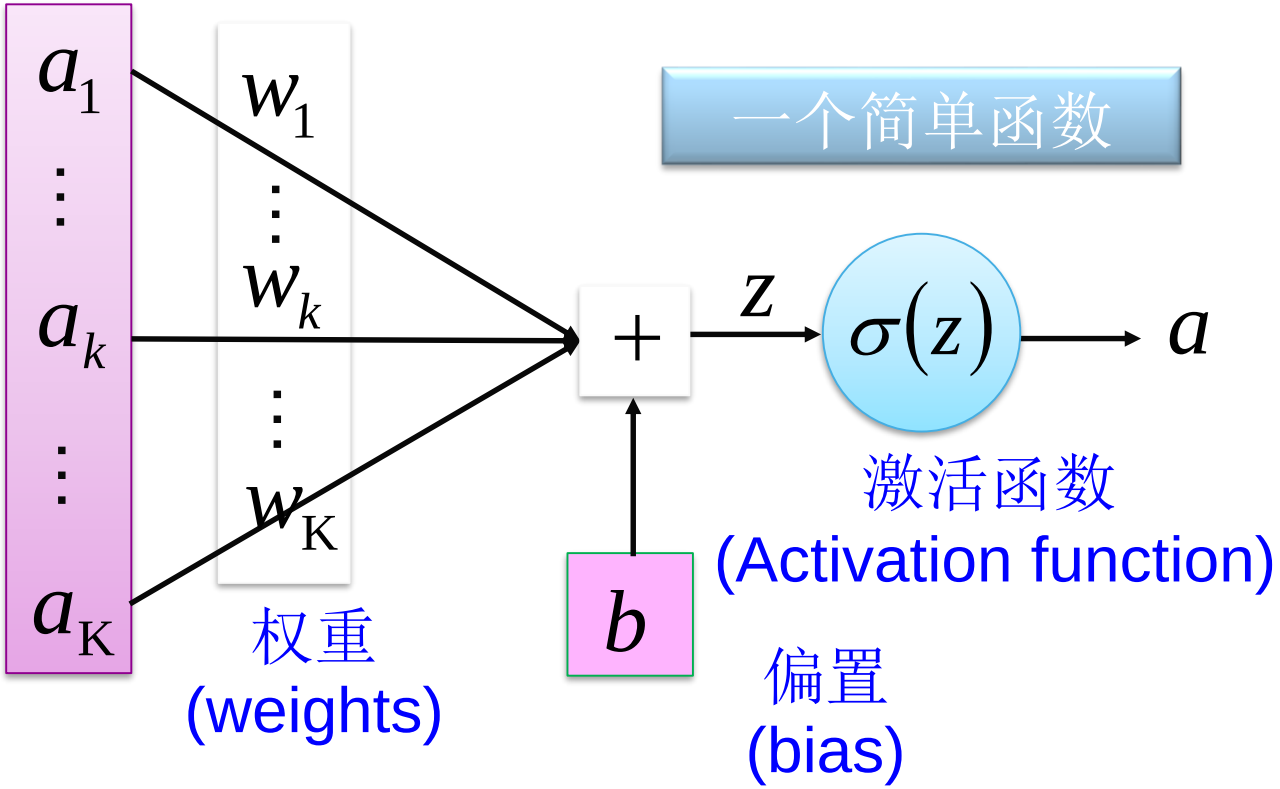
能解决异或问题的两层感知机

- 输出层与输入层之间的一层神经元，被称之为隐层或隐含层，隐含层和输出层神经元都是具有激活函数的功能神经元

人工神经网络（ANN）

神经元（Neuron）

$$z = a_1 w_1 + \cdots + a_k w_k + \cdots + a_K w_K + b$$

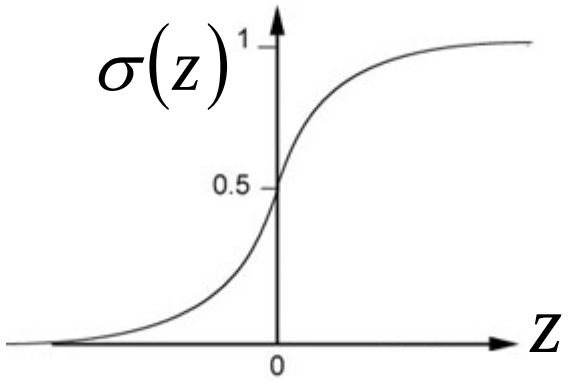


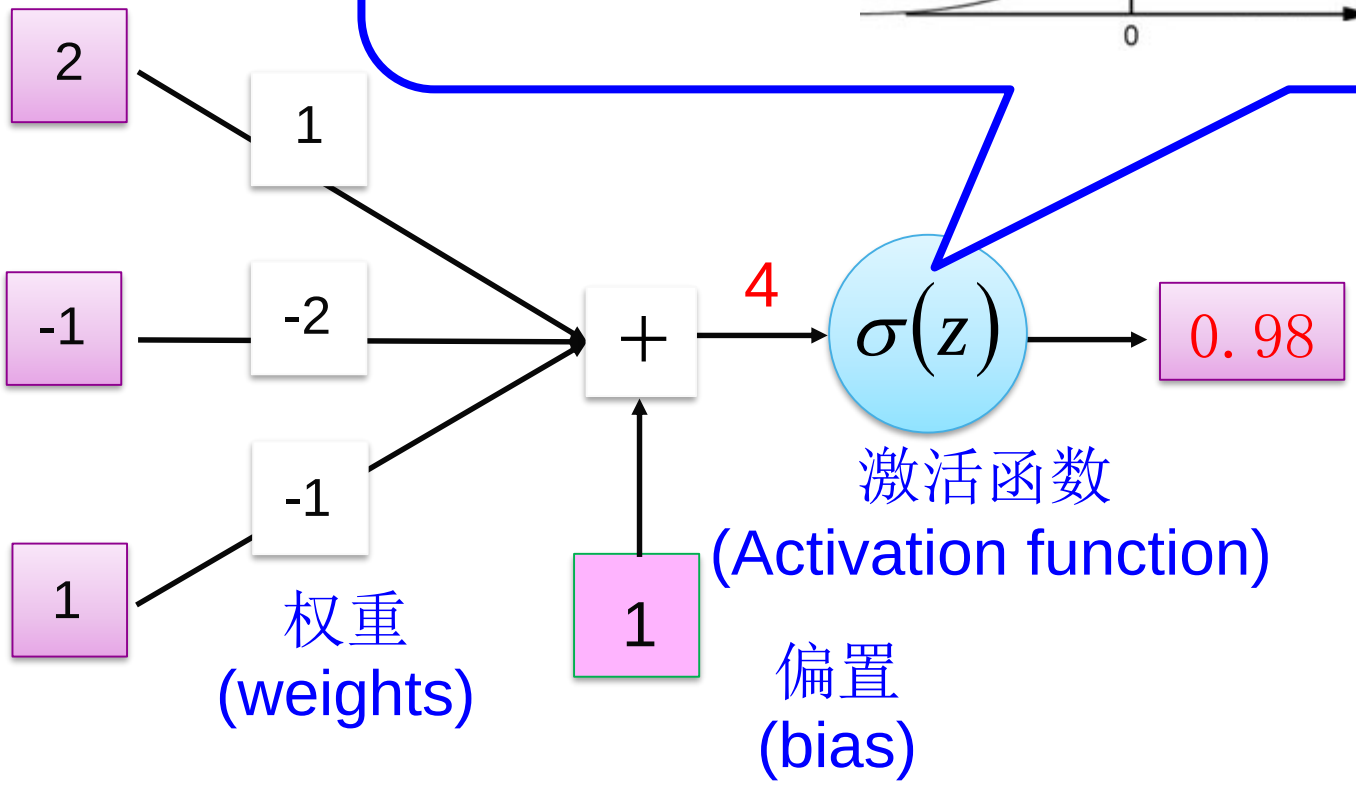
人工神经网络（ANN）

神经元（Neuron）

Sigmoid 函数

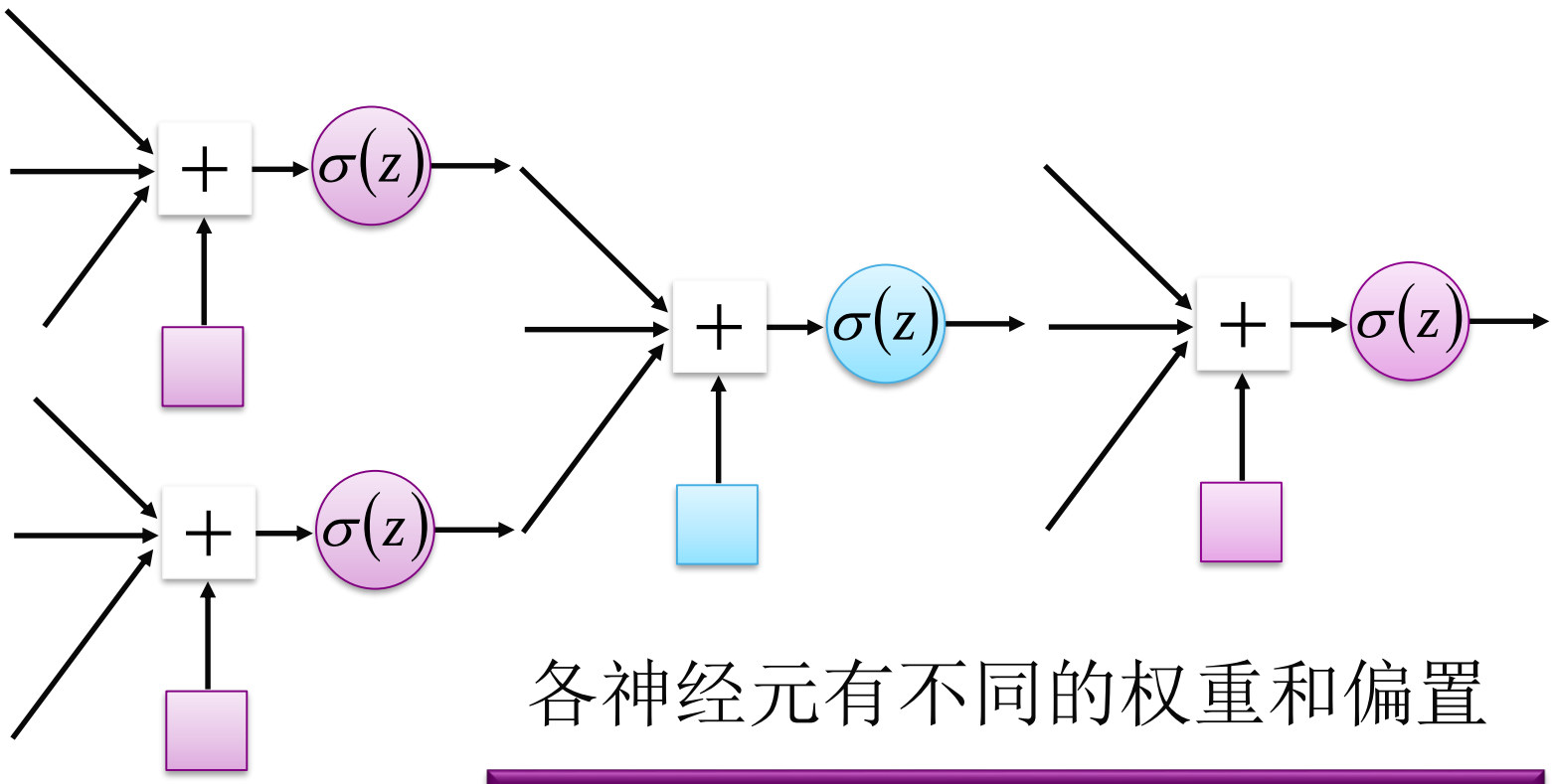
$$\sigma(z)=\frac{1}{1+e^{-z}}$$





人工神经网络（ANN）

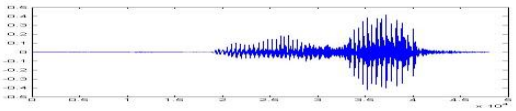
不同的连接将产生不同的网络结构



各神经元有不同的权重和偏置

权重和偏置是神经网络的参数θ

机器学习 ≈ 寻找一个函数

语音识别 $f($  $)=$ “How are you”

图像识别 $f($ $)=$ “Cat”

下围棋 $f($ $)=$ “5-5”

（下一步落子）

人机会话 $f($ “Hi” $)=$ “Hello”

（用户说） （系统回应）

深度学习三部曲

第一步：

构建神经网络

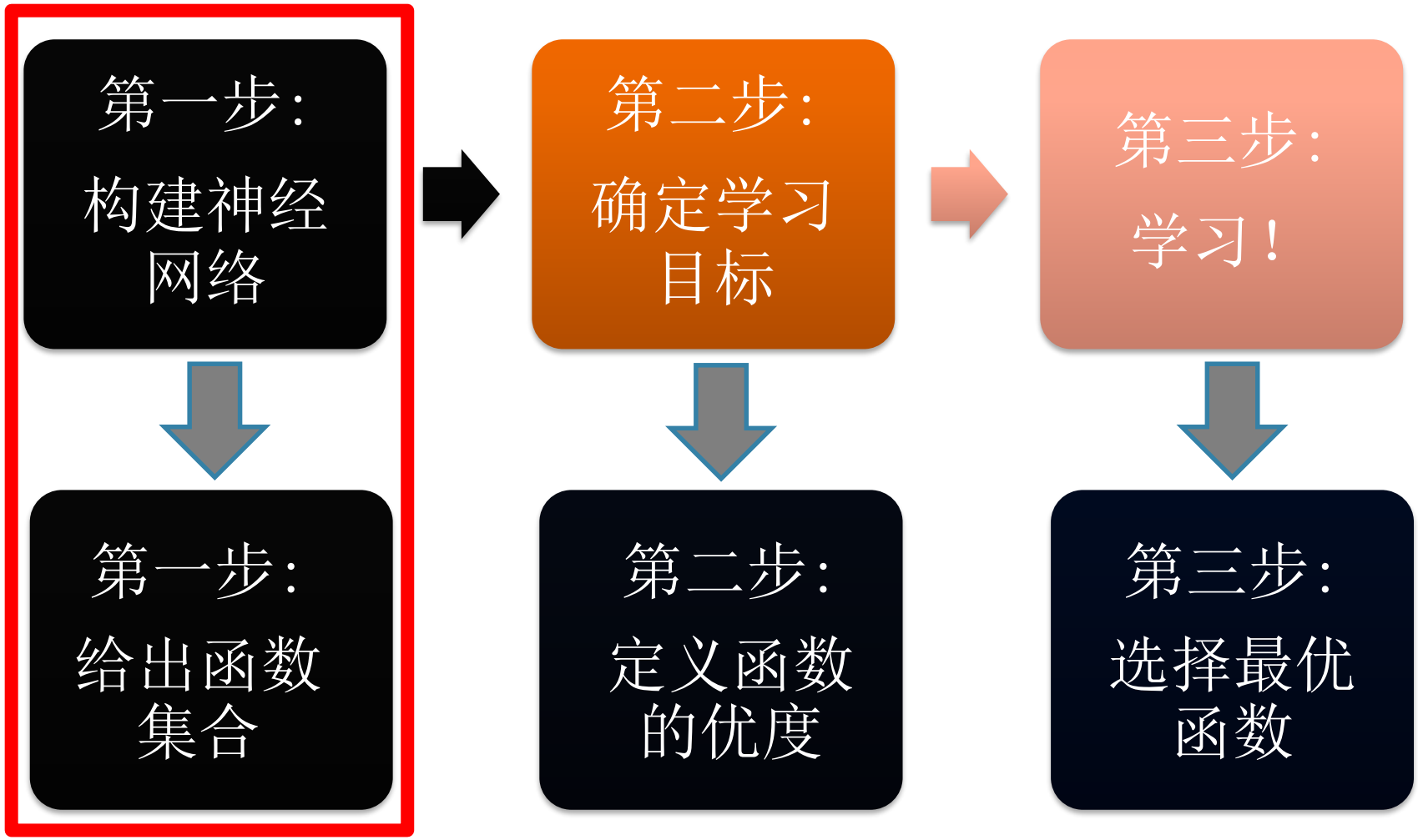
第二步：

确定学习目标

第三步：

学习！

深度学习三部曲


第一步：
构建神经网络
第二步：
确定学习目标
第三步：
学习！
第一步：
给出函数集合
第二步：
定义函数的优度
第三步：
选择最优函数

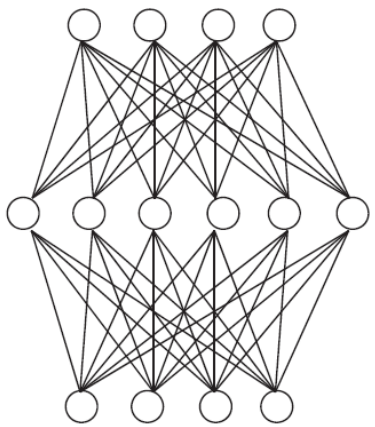
多层前馈神经网络

定义：每层神经元与下一层神经元全互联，神经元之间不存在同层连接也不存在跨层连接

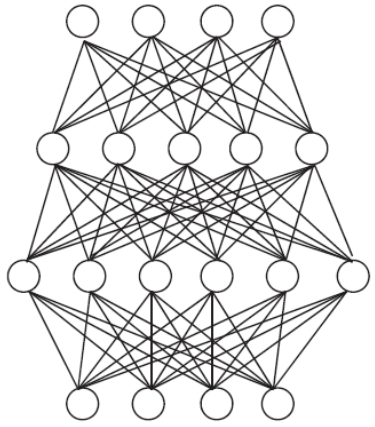
前馈：输入层接受外界输入，隐含层与输出层神经元对信号进行加工，最终结果由输出层神经元输出

学习：根据训练数据来调整神经元之间的“连接权”以及每个功能神经元的“阈值”

多层网络：包含隐层的网络



(a) 单隐层前馈网络

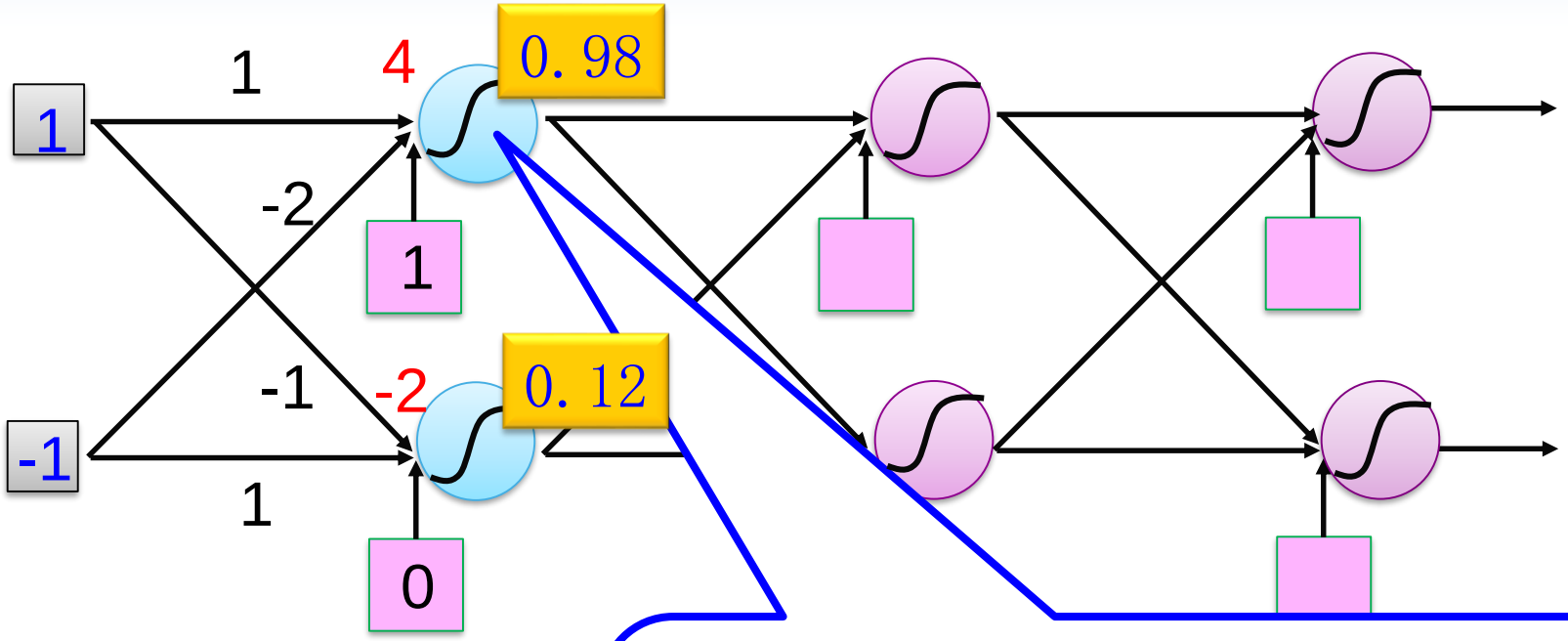


(b) 双隐层前馈网络

多层前馈网络有强大的表示能力（万有逼近性）：仅需一个包含足够多神经元的隐层，多层前馈神经网络就能以任意精度逼近任意复杂度的连续函数。

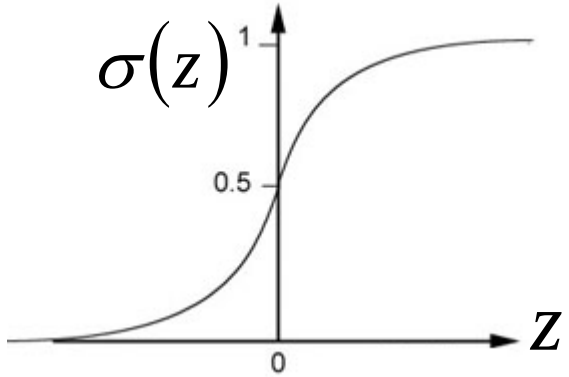
如何设置隐层神经元数？ → 试错法

全连接前馈网络

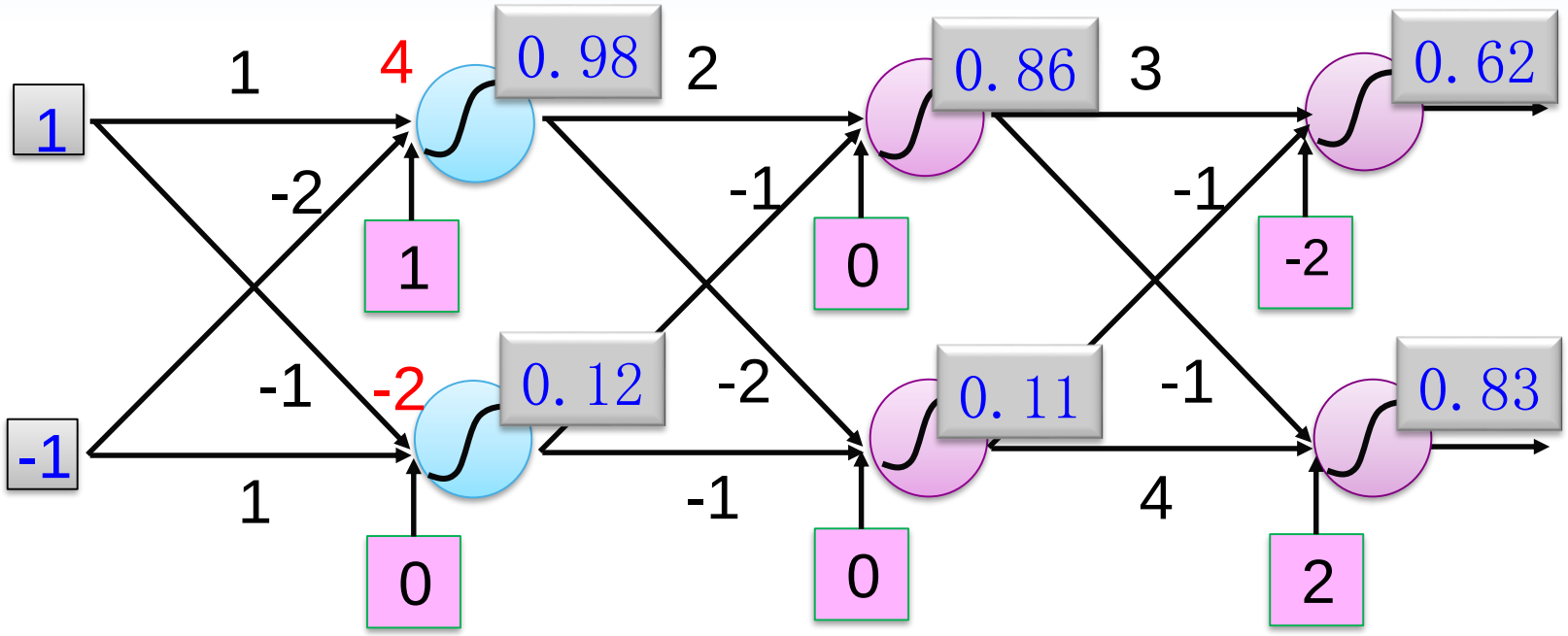


Sigmoid 函数

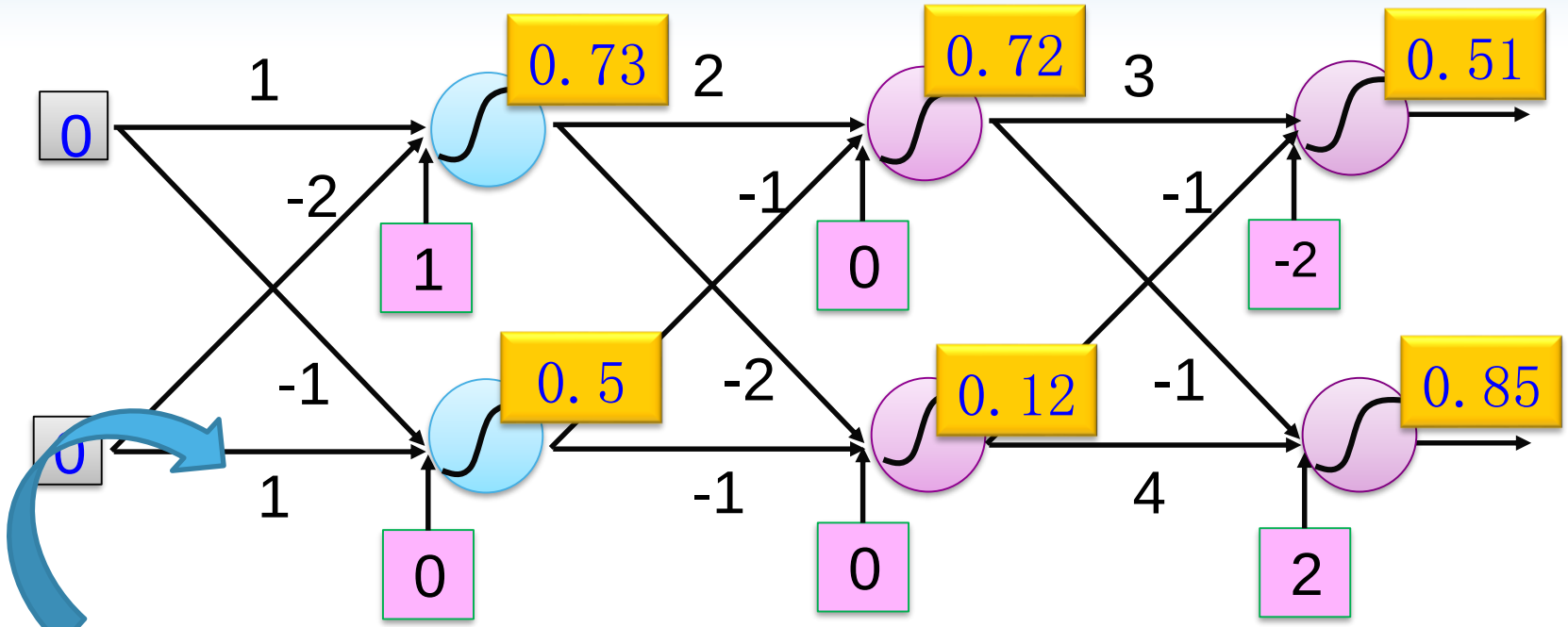
$$\sigma(z)=\frac{1}{1+e^{-z}}$$



全连接前馈网络



全连接前馈网络



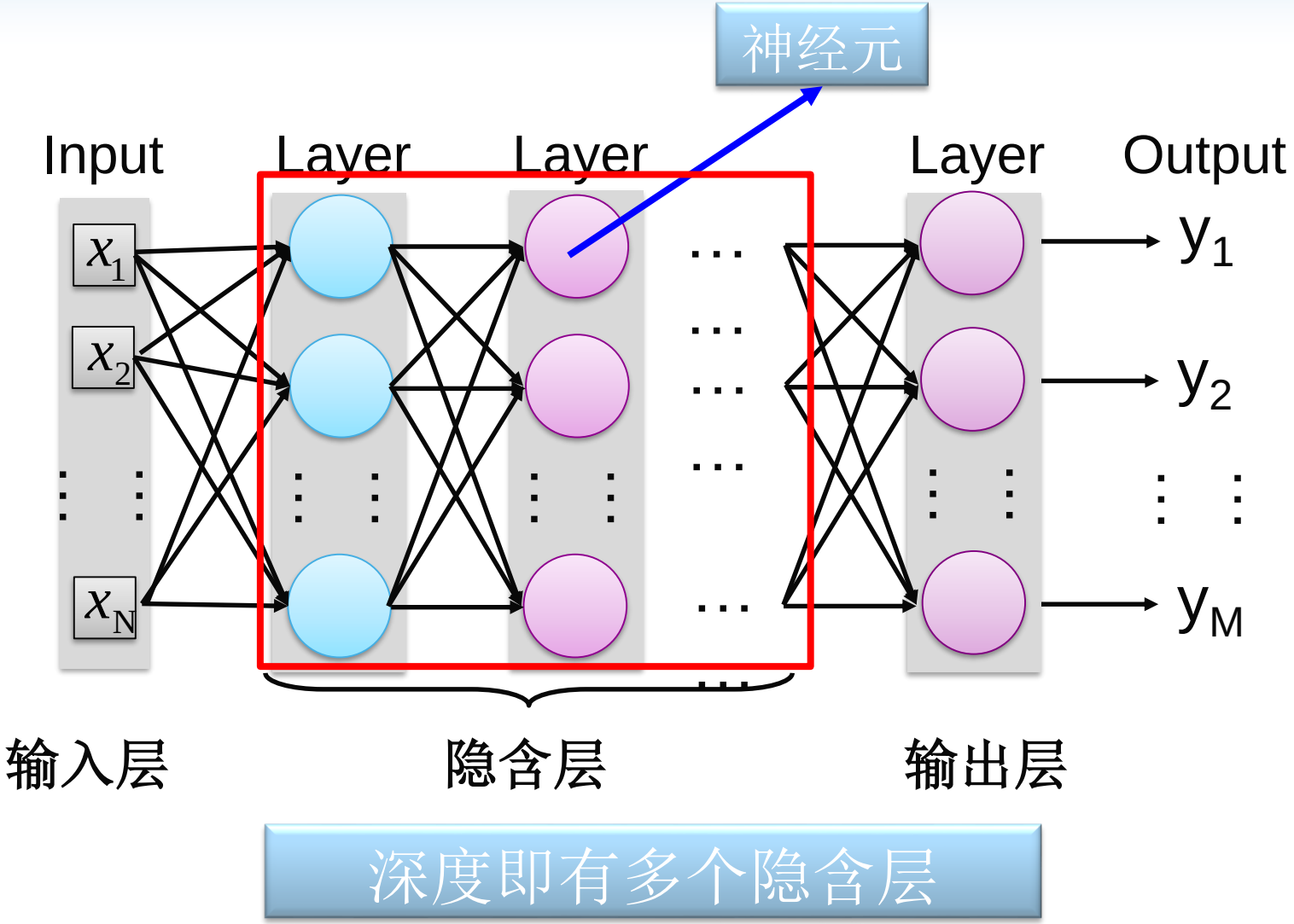
这是一个函数.

输入向量, 输出向量

$$f\left(\begin{bmatrix}1\\-1\end{bmatrix}\right)=\begin{bmatrix}0.62\\0.83\end{bmatrix}\quad f\left(\begin{bmatrix}0\\0\end{bmatrix}\right)=\begin{bmatrix}0.51\\0.85\end{bmatrix}$$

给定网络结构，就定义了一组函数

全连接前馈网络



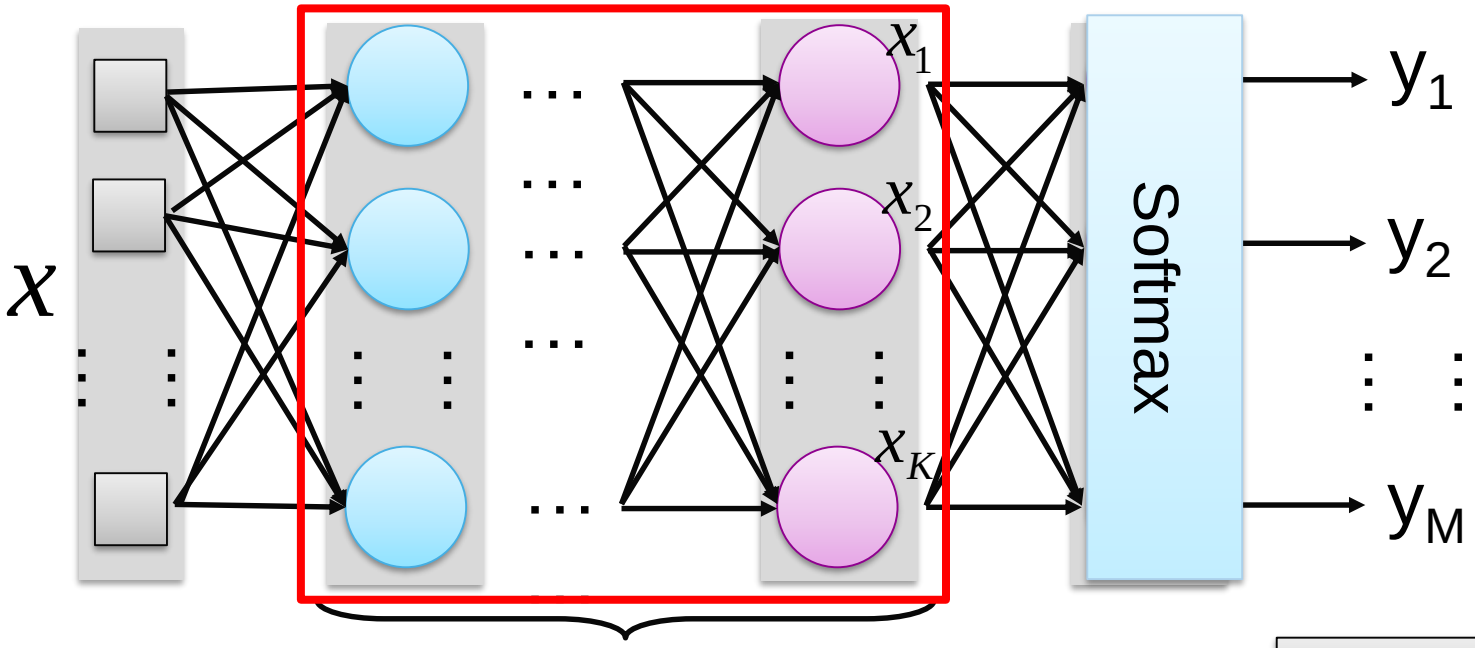
深度即有多个隐含层

输出层（多类分类器）

Probability:

- $1 > y_i > 0$
- $\sum_i y_i = 1$

特征抽取替代了特征工程



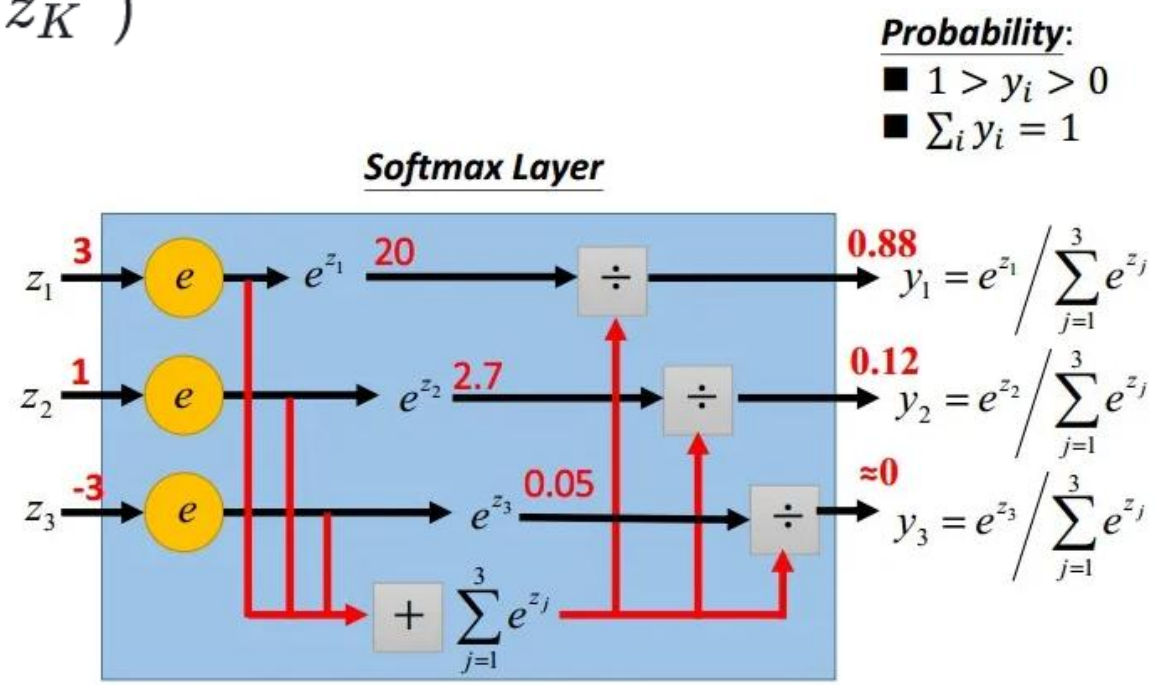
输入层　　隐含层　　输出层 = 多类分类器

softmax

softmax函数的基本形式是对一个K维的实数向量进行归一化，转变成另一个K维的实数向量，使得每一个元素的范围都在(0,1)之间，并且所有元素的总和为1。这使得softmax函数可以被解释为一个概率分布。

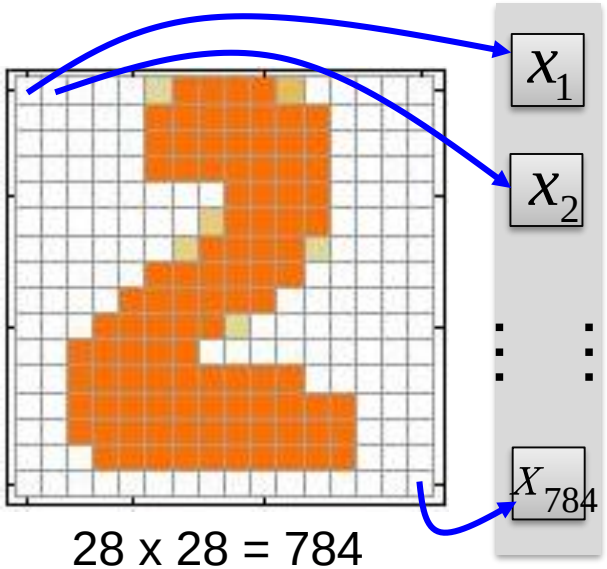
给定一个实数向量 $z = (z_1, z_2, \ldots, z_K)$

$$(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}$$



案例应用：手写数字识别

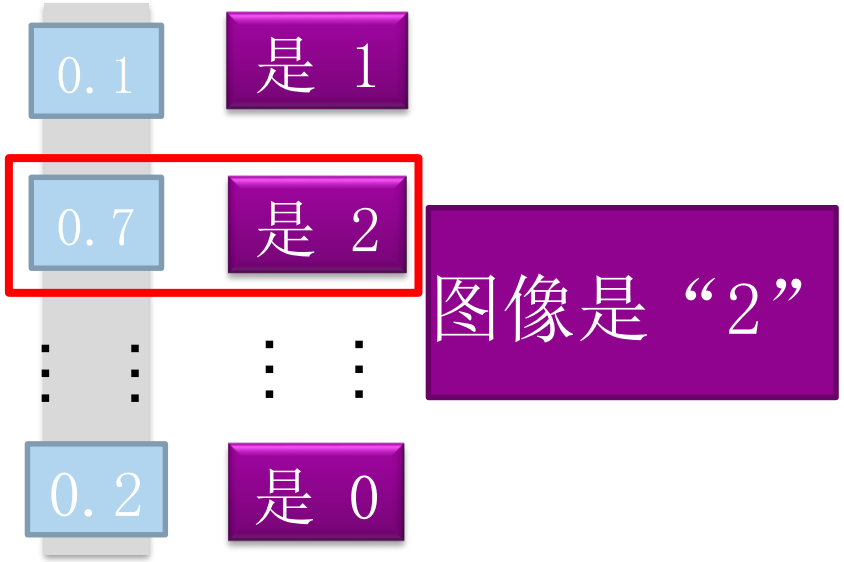
输入



28 x 28 = 784

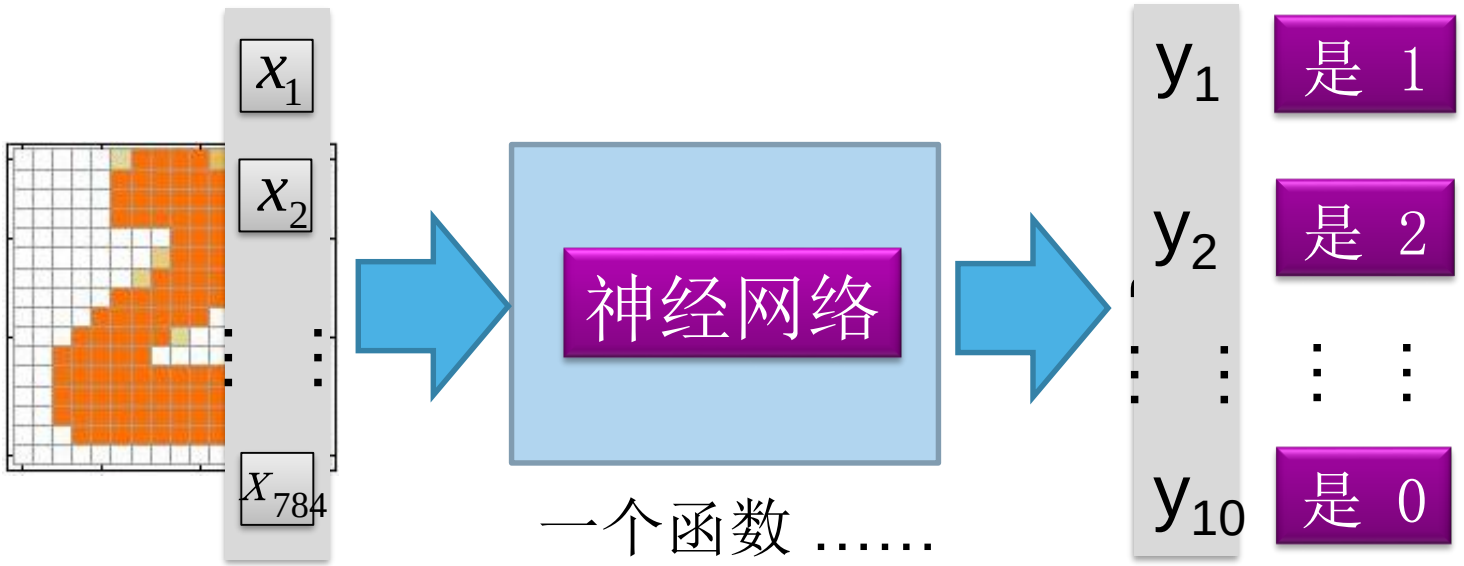
墨水 → 1
无墨水 → 0

输出



每一维表示预测为一个数字的概率

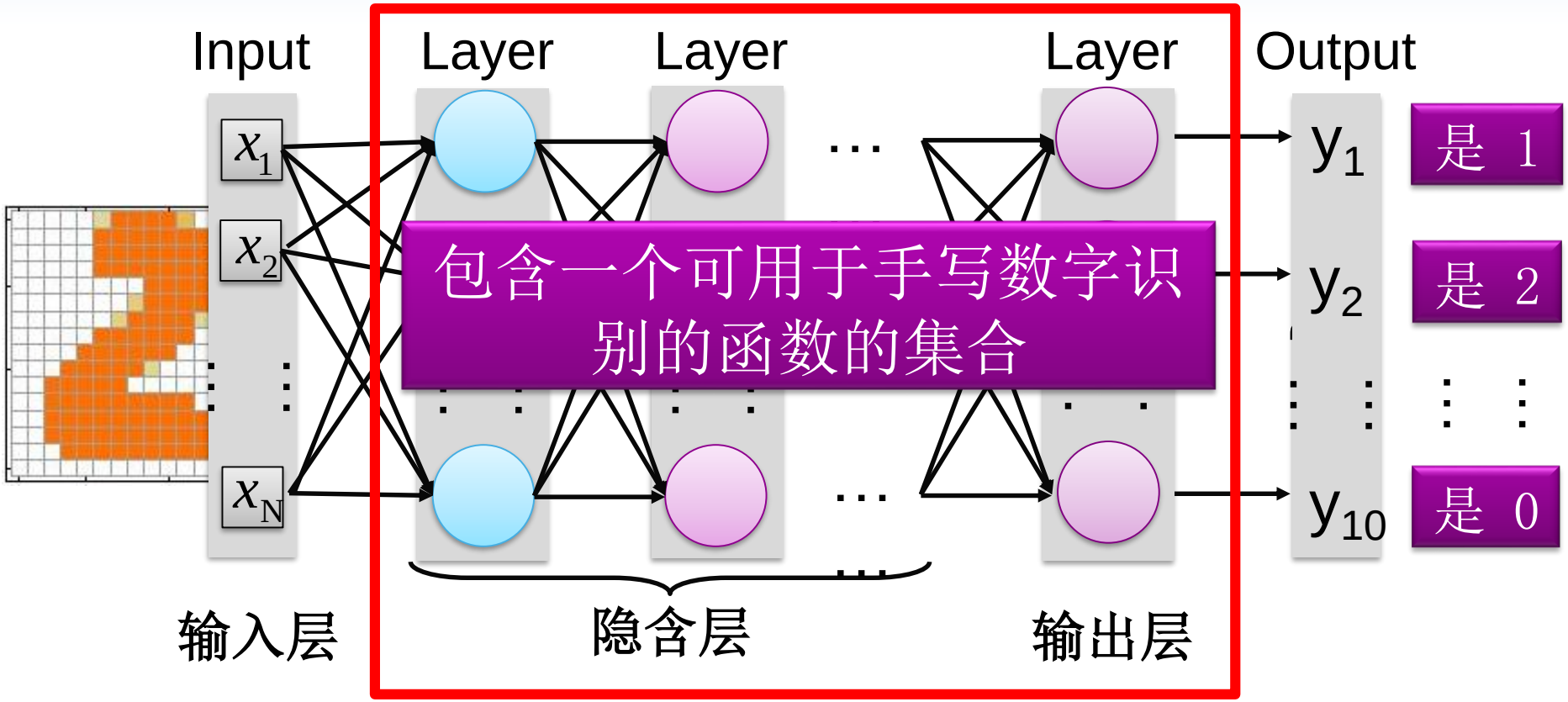
案例应用：手写数字识别



输入:
784维向量

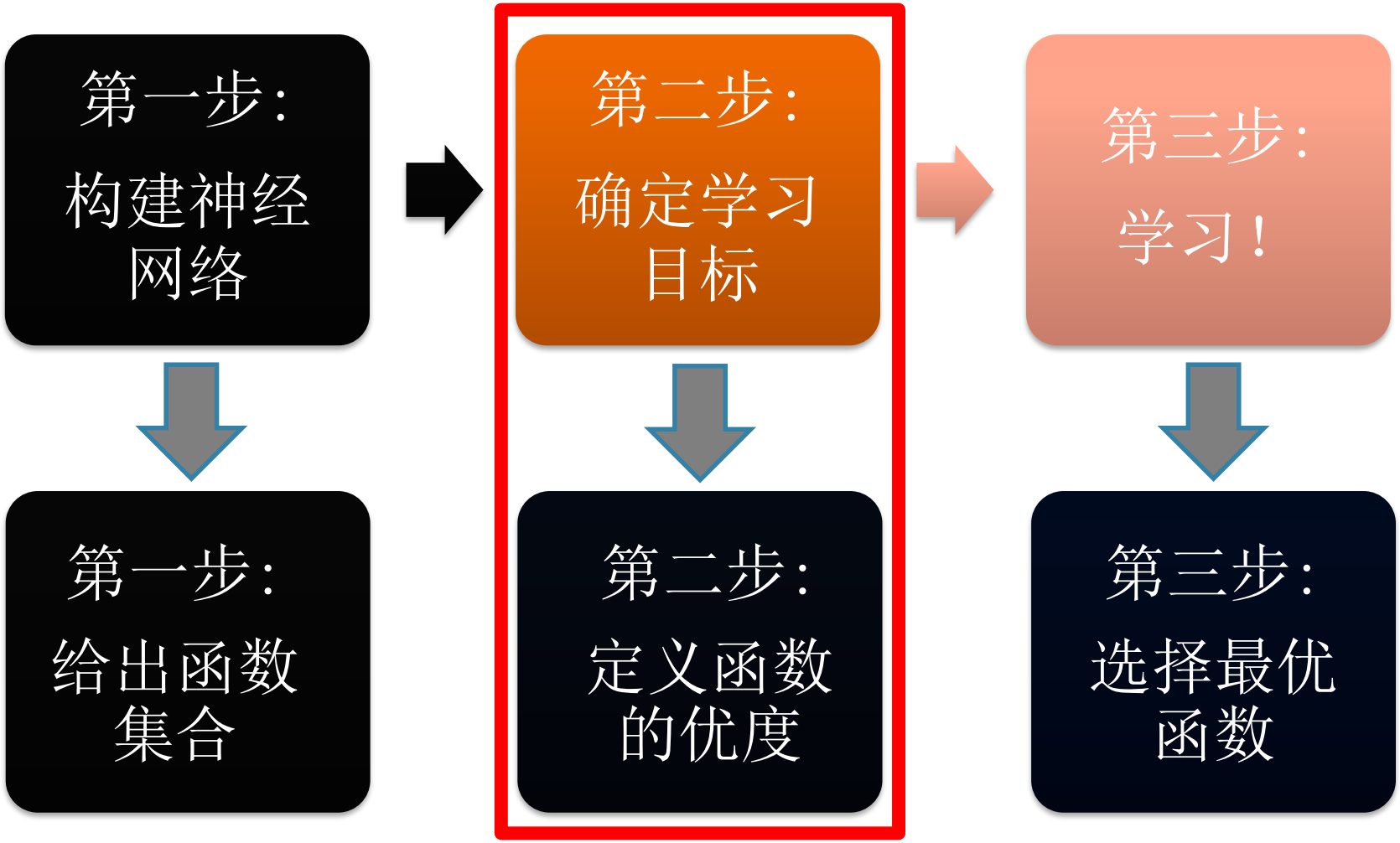
输出:
10维向量

案例应用：手写数字识别



需要构建一个神经网络结构以包含一个好的可用于手写数字识别的函数.

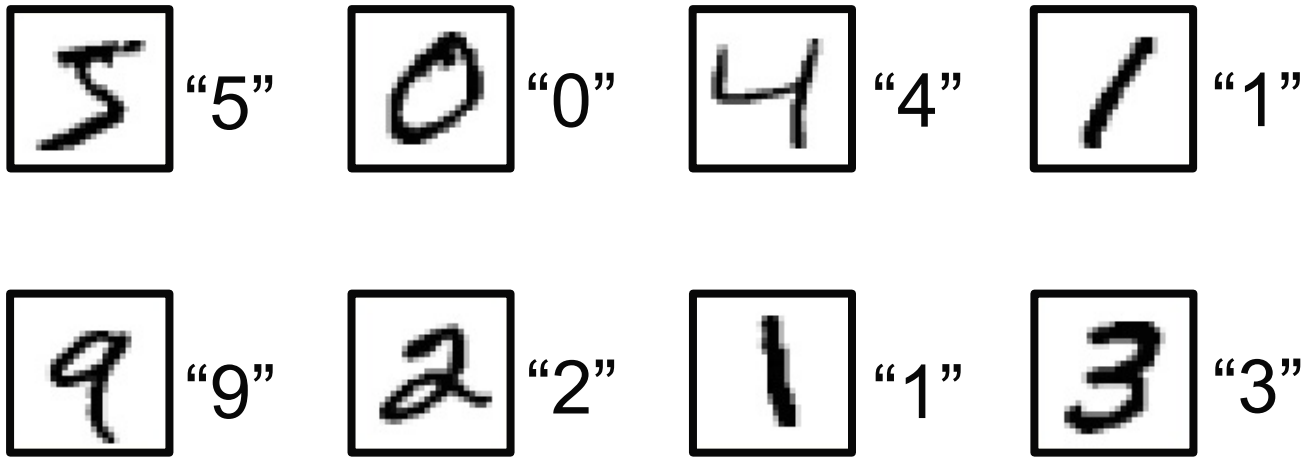
深度学习三部曲


第一步：
构建神经网络
第二步：
确定学习目标
第三步：
学习！
第一步：
给出函数集合
第二步：
定义函数的优度
第三步：
选择最优函数

14.1 深度学习发展简史
14.2 多层感知机
14.3 损失函数
14.4 优化算法
14.5 案例：手写数字识别
14.6 深度学习技巧
14.7 卷积神经网络
14.8 案例：图像识别

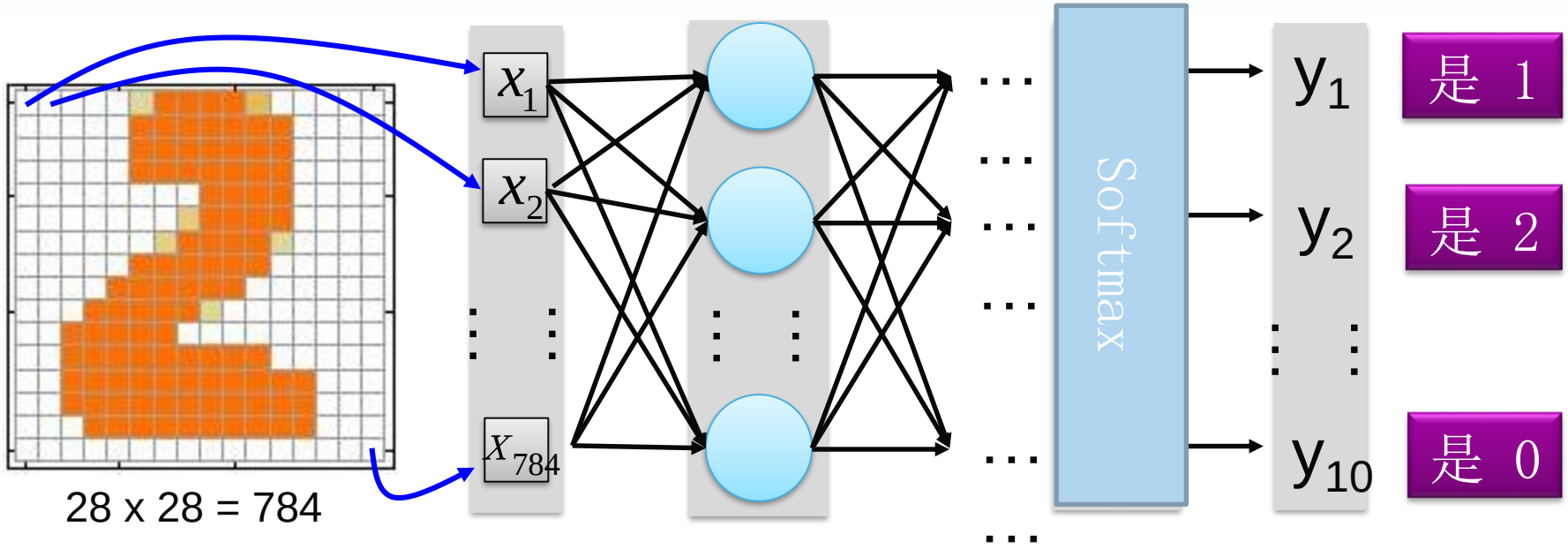
训练数据

准备训练数据：图像及其标签



学习目标定义在训练数据之上

学习目标



28 x 28 = 784

墨水 → 1

无墨水 → 0

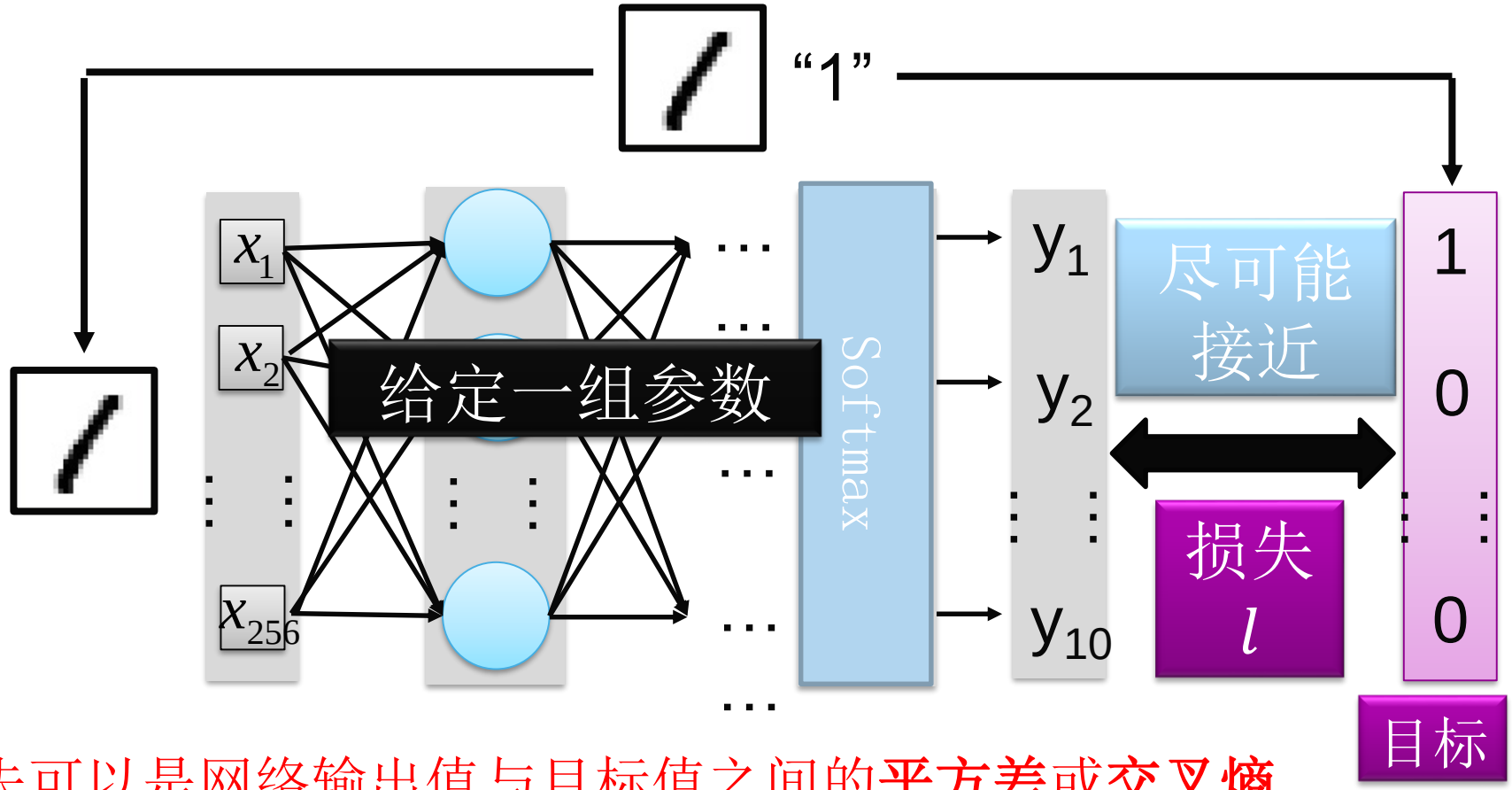
学习目标是: ……

输入: → y_1 值最大

输入: → y_2 值最大

损失

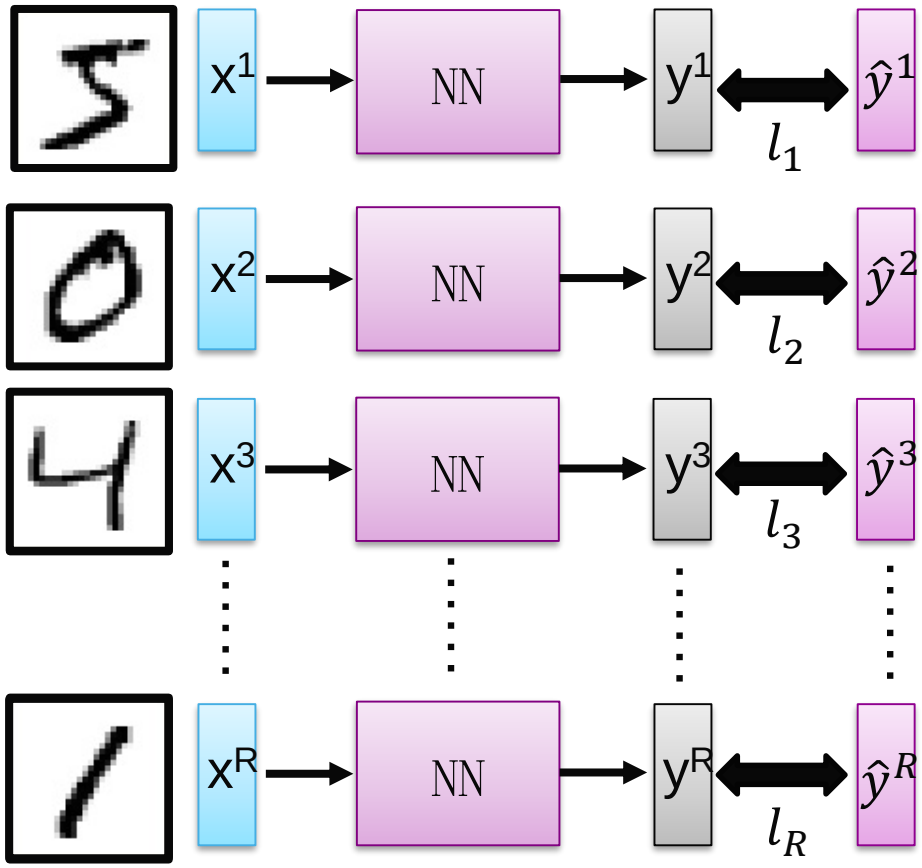
一个好的函数应该使所有的训练样例产生的损失尽可能小.



损失可以是网络输出值与目标值之间的**平方差**或**交叉熵**

整体损失

对于所有的训练数据…



整体损失:

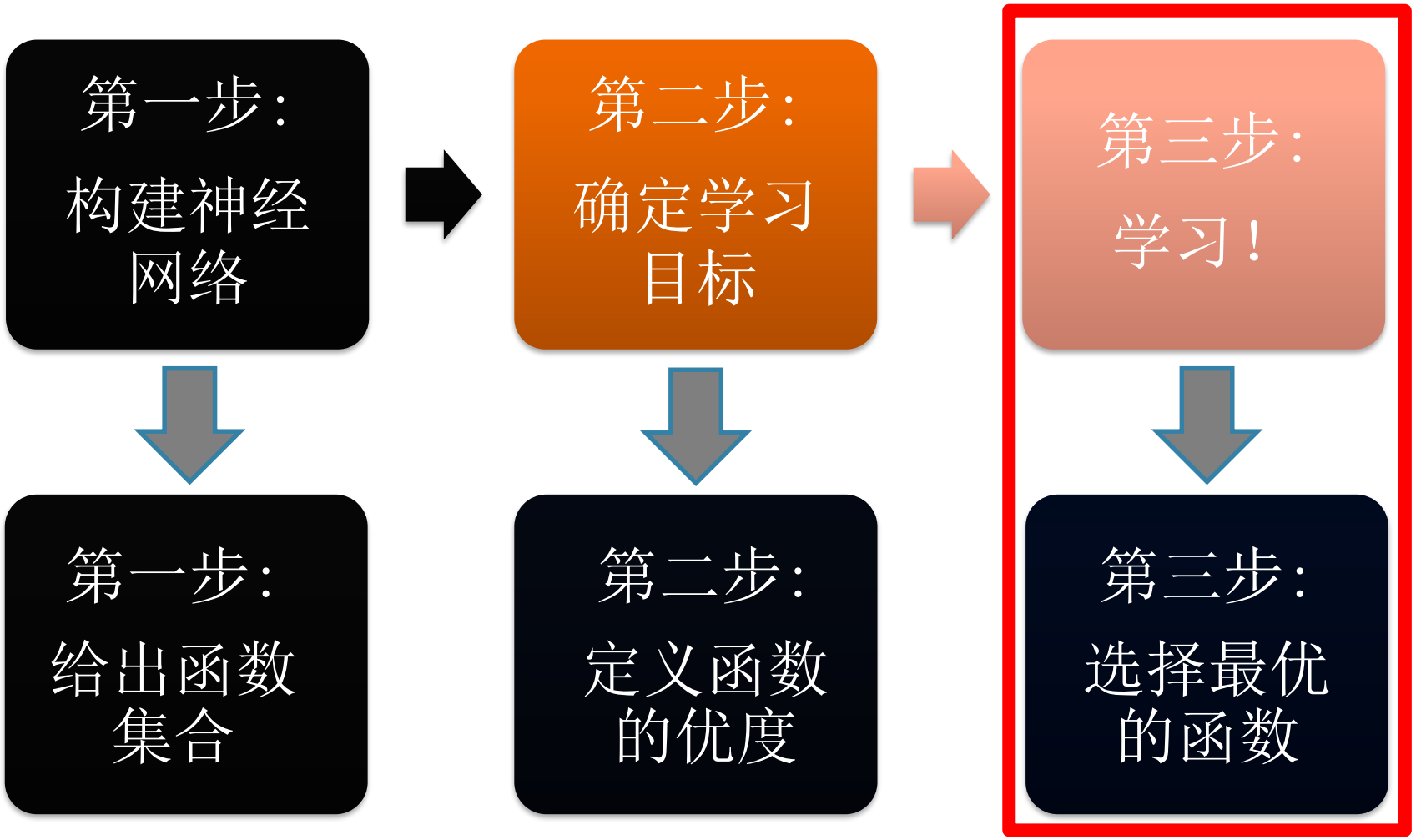
$$L = \sum_{r=1}^{R} l_r$$

尽可能小

在函数集中找到使整体损失L最小的一个函数

找到能最小化整体损失L的一组网络参数 $\boldsymbol{\theta}^*$

深度学习三部曲


第一步：
构建神经网络
第二步：
确定学习目标
第三步：
学习！
第一步：
给出函数集合
第二步：
定义函数的优度
第三步：
选择最优的函数

14.1 深度学习发展简史
14.2 多层感知机
14.3 损失函数
14.4 优化算法
14.5 案例：手写数字识别
14.6 深度学习技巧
14.7 卷积神经网络
14.8 案例：图像识别

如何选择最优的函数

找到能最小化整体损失L的一组网络参数 $\boldsymbol{\theta}^*$

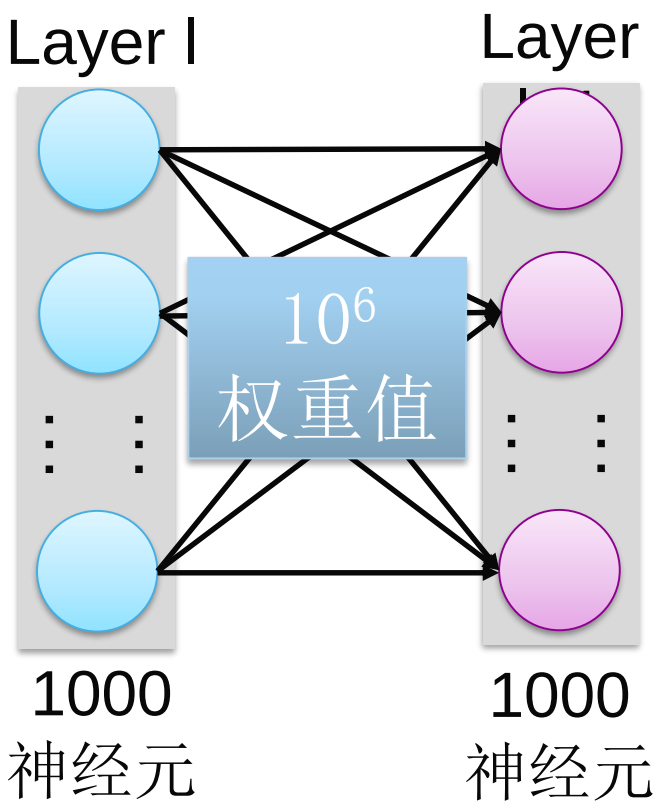
枚举所有可能的参数值

网络参数 θ

$= \{w_1, w_2, \cdots, b_1, b_2, b_3, \cdots\}$

成百[illegible]个参数

例如, 语音识别: 8层, 1000个神经元/层



梯度下降法

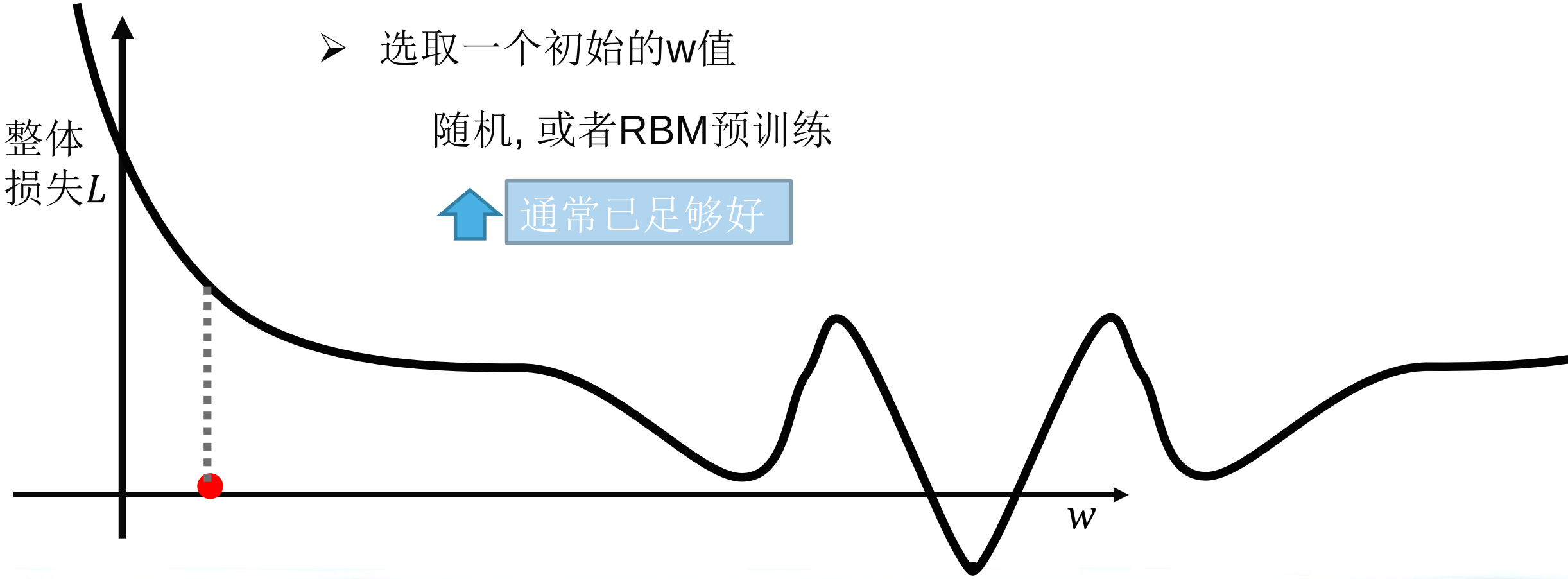
网络参数 θ
$= \{w_1, w_2, \cdots, b_1, b_2, \cdots\}$

找到能最小化整体损失L的一组网络参数 $\boldsymbol{\theta}^*$

- 选取一个初始的w值

  随机, 或者RBM预训练

  通常已足够好

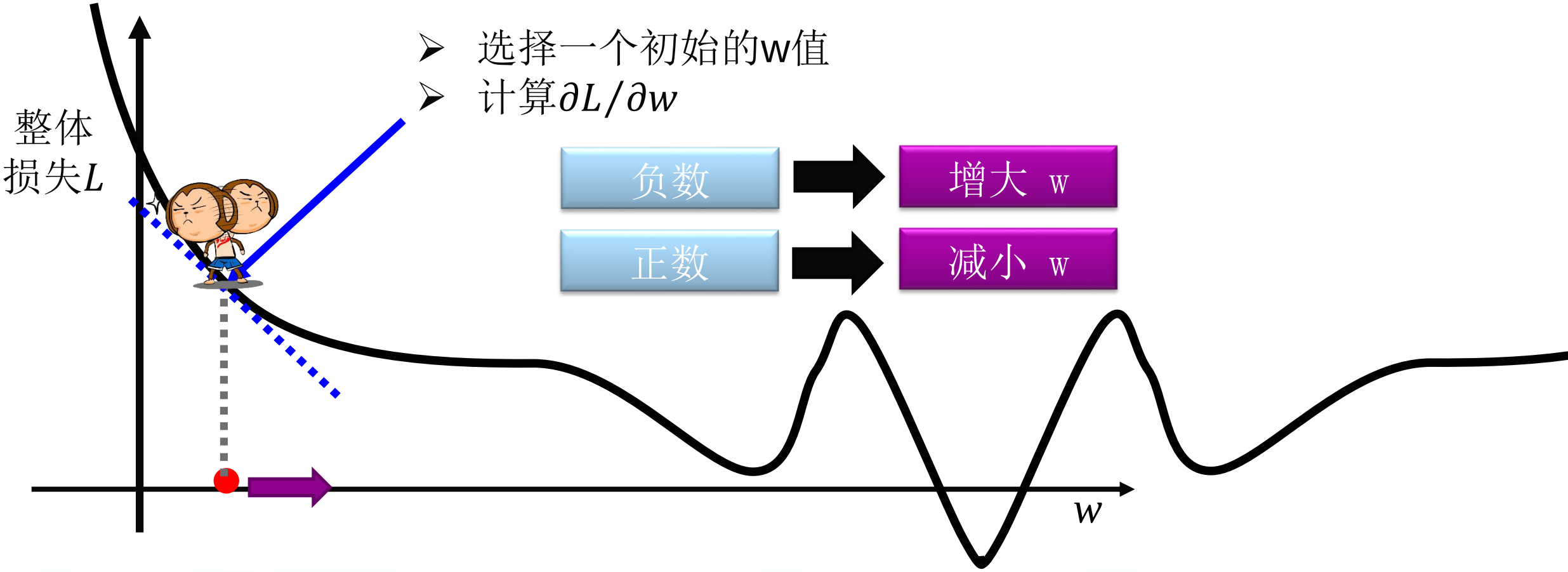


梯度下降法

网络参数 θ
$= \{w_1, w_2, \cdots, b_1, b_2, \cdots\}$

找到能最小化整体损失L的一组网络参数 $\boldsymbol{\theta}^*$

- 选择一个初始的w值
- 计算$\partial L/\partial w$



负数 → 增大 w

正数 → 减小 w

梯度下降法

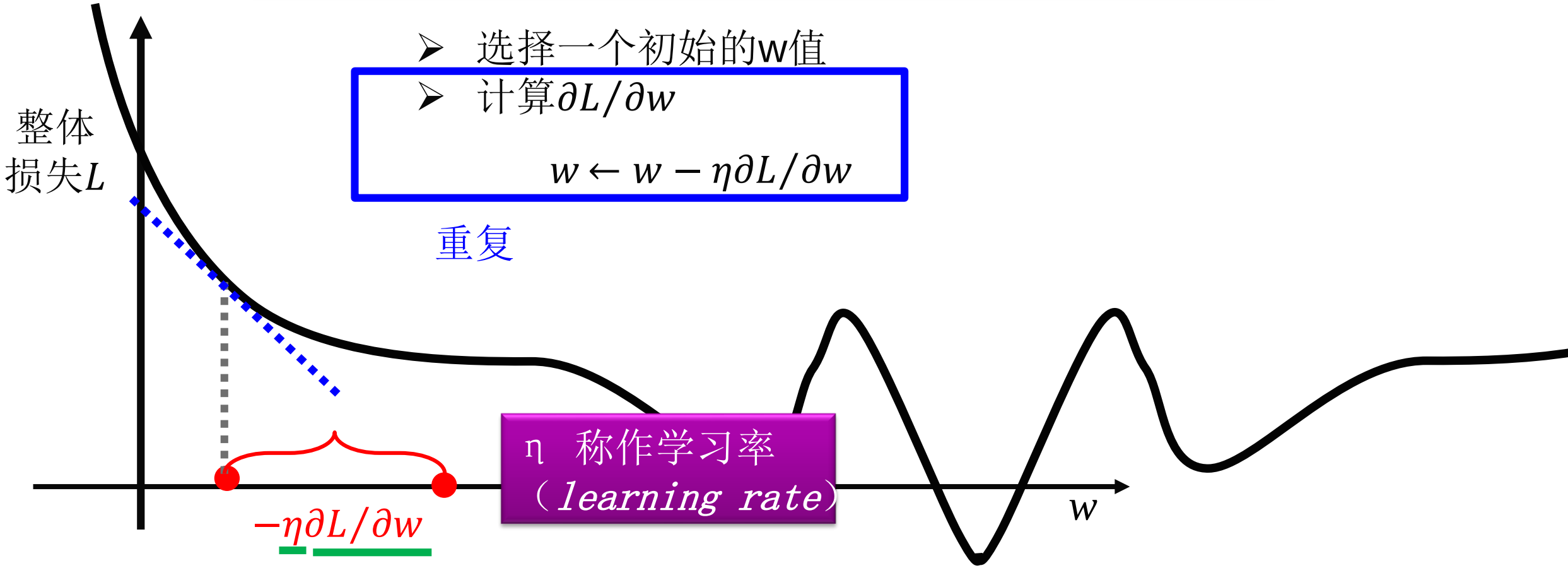
网络参数 θ
$= \{w_1, w_2, \cdots, b_1, b_2, \cdots\}$

找到能最小化整体损失L的一组网络参数 $\boldsymbol{\theta}^*$

- 选择一个初始的w值
- 计算$\partial L/\partial w$

$$w \leftarrow w - \eta \partial L/\partial w$$

重复



η 称作学习率
（*learning rate*）

梯度下降法

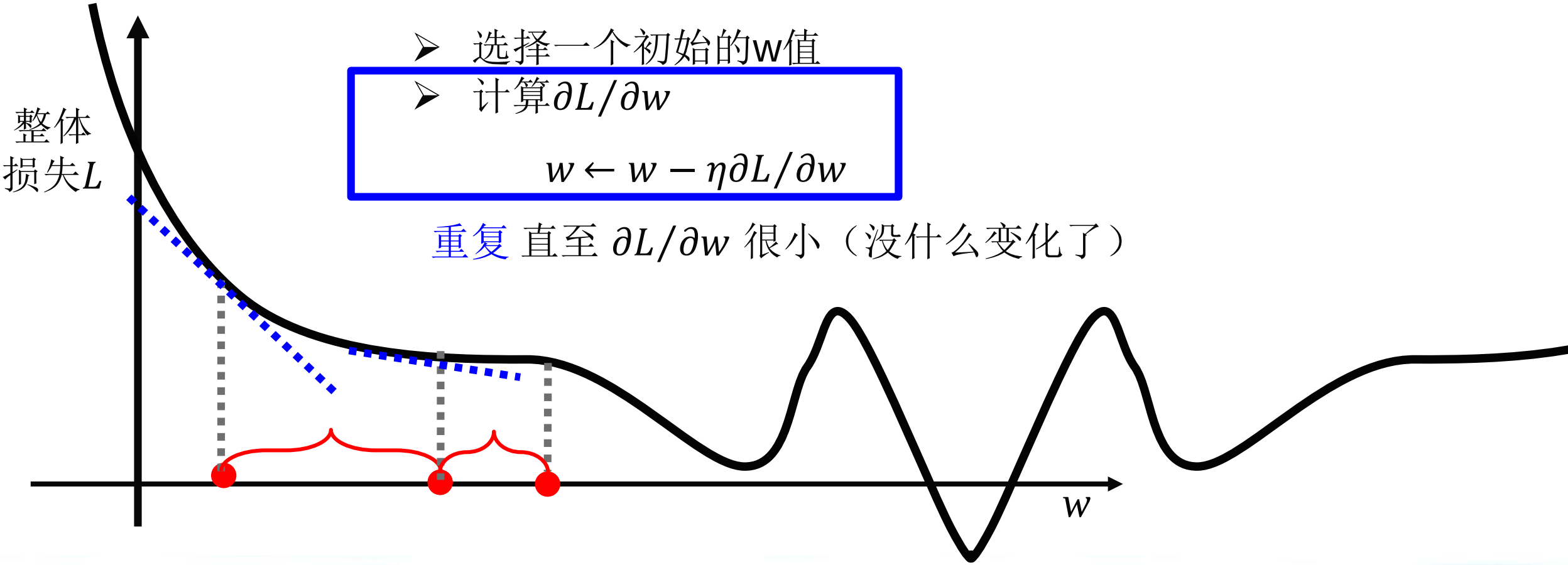
网络参数 θ
$= \{w_1, w_2, \cdots, b_1, b_2, \cdots\}$

找到能最小化整体损失L的一组网络参数 $\boldsymbol{\theta}^*$

- 选择一个初始的w值
- 计算$\partial L/\partial w$

$$w \leftarrow w - \eta \partial L/\partial w$$

重复 直至 $\partial L/\partial w$ 很小（没什么变化了）



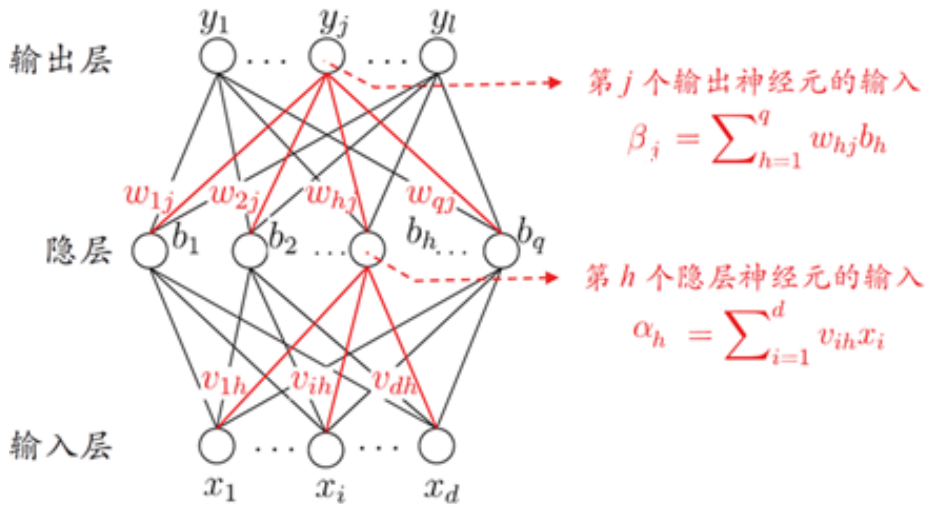
误差逆传播算法

误差逆传播算法（Error BackPropagation，简称BP）是最成功的训练多层前馈神经网络的学习算法.

- 给定训练集$D=\{(\boldsymbol{x}_i,y_i)\},\boldsymbol{x}_i\in R^d,y_i\in R^l,(i=1,2,\cdots,m)$，即输入实例由$d$个属性描述，输出$l$维实值向量.
- 给定一个拥有d个输入神经元，l个输出神经元，q个隐层神经元的多层前向前馈网络结构.
- 记号：

θ_j：输出层第j个神经元阈值；

γ_h：隐含层第h个神经元阈值；

v_{ih}：输入层与隐层神经元之间连接权重；

w_{hj}：隐层与输出层神经元之间连接权重；